

DATA STRUCTURES & ALGORITHMS II
XI'AN JIAOTONG UNIVERSITY · FINAL EXAM HANDBOOK

数据结构与 算法 II 考前突击手册

基于 2026 年 B 卷真题反向工程

本手册基于西安交通大学《数据结构与算法 II》课程 10 章节核心知识点编制，涵盖**知识梳理**、**公式速记**、**复杂度对照**、**易错点速查**、**分板块模拟题**四大模块。针对已粗略看过 PPT、能完成 60-70% 选择题的同学设计，覆盖 300+ 道题目与 25+ 道代码设计题，帮助在 1-2 天内系统巩固应试能力。

COURSE 数据结构与算法 II · 期末考试
SCOPE 10 章节 · 5 模拟板块 · 真题附录
LEVEL 考前 1-2 天突击复习专用

目 录

使用说明	2
第一部分 知识点全面梳理	3
第1章 绪论	3
第2章 线性表	5
第3章 栈和队列	7
第4章 串	9
第5章 数组和广义表	11
第6章 树和二叉树	13
第7章 图	18
第9章 查找	21
第10章 排序	25
第二部分 公式速记卡与对照表	30
第三部分 分板块模拟题	33
第四部分 考前1小时冲刺页 + 真题B卷全解附录	64

使用说明

本手册基于西安交通大学《数据结构与算法II》2026年B卷真题反向工程编制，覆盖10个章节的核心知识点、高频考点与典型题型。手册定位为"考前突击"，并非完整教材，而是帮助已粗略看过PPT的同学在1-2天内系统巩固。

建议使用方法：①先快速浏览第一部分知识梳理（约2-3小时），建立整体框架；②针对薄弱章节，做第三部分对应板块的模拟题（每板块30题+代码题），做完后对照解析；③考前1小时，重点看第二部分公式速记卡、复杂度对照表、易错点速查表，以及最后的冲刺页。

符号约定

- 【提示】蓝色框：补充说明或记忆技巧
- 【注意】黄色框：常见易错点
- 【正确】绿色框：标准答案或正确做法
- 【错误】红色框：典型错误示例
- 代码块：左侧蓝色竖线，使用SarasaMonoSC等宽字体
- Mermaid流程图：自动渲染的算法原理图

章节说明

原课程包含10章，第8章（动态存储管理）不在期末考试范围，故本手册跳过。第9、10章（查找、排序）为重难点，分别占用较多篇幅。所有模拟题均按真题题型分布：选择题、填空题、简答题、算法设计题，每板块题目数不少于30题，全手册代码题合计超过20题。

第一部分 知识点全面梳理

本部分按教材章节顺序系统梳理数据结构与算法的核心知识点。每章包含：基本概念定义、关键性质与公式、存储结构对比、核心算法的C语言实现（或伪代码）、典型例题解析、易错点提示。建议结合PPT同步复习，重点掌握标有【注意】的内容。

第1章 绪论

1.1 基本概念与术语

数据 (Data) 是客观事物符号表示的集合，是计算机程序处理的符号总称。**数据元素 (Data Element)** 是数据的基本单位，在程序中作为一个整体处理。一个数据元素可由若干个**数据项 (Data Item)** 组成，数据项是数据不可分割的最小单位。例如，学生记录是一个数据元素，而学号、姓名、年龄等是该数据元素的数据项。

数据对象 (Data Object) 是性质相同的数据元素的集合，是数据的一个子集。**数据结构 (Data Structure)** 是相互之间存在一种或多种特定关系的数据元素的集合，包括两个层面：**逻辑结构**（数据元素间的关系）和**存储结构**（数据在计算机中的表示）。

1.2 逻辑结构四类

数据元素之间的逻辑关系分为四类基本结构：

- **集合**：数据元素除同属一个集合外无其他关系
- **线性结构**：元素间一对一关系（线性表、栈、队列、串、数组、广义表）
- **树形结构**：元素间一对多关系（树、二叉树）
- **图状结构**：元素间多对多关系（有向图、无向图）

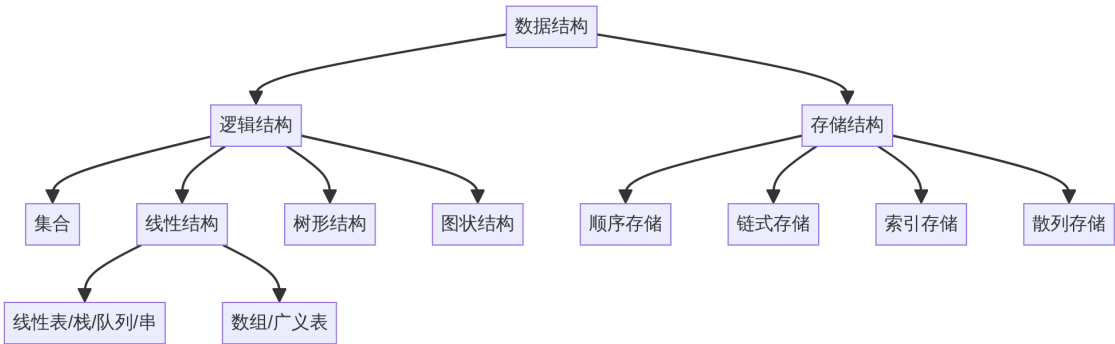


图1-1 数据结构的逻辑分类

1.3 存储结构四类

数据的存储结构（物理结构）是逻辑结构在计算机中的实现，分四类：

存储结构	特点	适用场景
顺序存储	元素存于地址连续的存储单元，逻辑相邻则物理相邻	线性表（数组）、栈、队列
链式存储	元素存于任意位置，通过指针指示逻辑关系	链表、二叉链表
索引存储	建立附加索引表，通过索引访问数据	数据库、分块查找
散列存储	通过哈希函数直接计算存储地址	哈希表

1.4 算法与算法分析

算法（Algorithm）是为了解决某类问题而描述的有限操作序列。算法的5个特性：**有穷性**（有限步结束）、**确定性**（每步有明确含义）、**可行性**（操作可通过基本运算实现）、**输入**（0或多个）、**输出**（1或多个）。

时间复杂度用大O记号表示算法执行时间与问题规模n的增长关系。分析方法：

- **频度法**：计算所有语句执行次数之和，取最高阶项，忽略系数
- **求和法**：对循环嵌套用求和公式，如 $\sum_{i=1}^n i = n(n+1)/2 \rightarrow O(n^2)$
- **递推法**：对递归算法列递推关系，如 $T(n) = 2T(n/2) + n \rightarrow O(n \log n)$

【提示】 常见时间复杂度排序： $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$ 。考试常考的递归复杂度：二分查找 $O(\log n)$ ，归并排序 $O(n \log n)$ ，快排最坏 $O(n^2)$ 平均 $O(n \log n)$ 。

1.5 典型例题

例1 求以下算法的时间复杂度：

```
for (i = 1; i < n; i++) {
    for (j = 1; j < i; j++) {
        x++; // 基本操作
    }
}
```

解：内层循环执行次数为 $\sum_{i=1}^{n-1} (i-1) = \sum_{k=0}^{n-2} k = (n-1)(n-2)/2$ ，展开后最高阶为 $n^2/2$ ，故时间复杂度为 $O(n^2)$ 。

例2（真题填空题改编）某算法语句执行频度之和为 $10n^2 \log n + 100n^2m/1000$ ，求时间复杂度。

解：忽略常数系数，保留最高阶项。若m视为与n无关的常数，则化简为 $O(n^2 \log n)$ 。若m与n同阶，则两项均保留，写作 $O(n^2 \log n + n^2m)$ 或 $O(n^2 \max(\log n, m))$ 。

第2章 线性表

2.1 线性表的定义与特点

线性表是 $n(n \geq 0)$ 个数据元素的有限序列。当 $n=0$ 时为空表；非空线性表特点：存在唯一一个被称为"第一个"的元素（无前驱）；存在唯一一个被称为"最后一个"的元素（无后继）；除第一个外，每个元素有且仅有一个直接前驱；除最后一个外，每个元素有且仅有一个直接后继。

2.2 顺序表（顺序存储）

顺序表用一组地址连续的存储单元依次存储线性表元素。设元素占用 c 个存储单元，首元素地址为 $LOC(a_1)$ ，则第 i 个元素地址： $LOC(a_i) = LOC(a_1) + (i-1) \times c$ 。

插入操作：在第 $i(1 \leq i \leq n+1)$ 位置插入新元素，需将第 n 到第 i 个元素后移一位。

移动次数 = $n - i + 1$ 。等概率情况下平均移动次数 = $\sum_{i=1}^{n+1} (n-i+1)/(n+1) = n/2$ ，时间复杂度 $O(n)$ 。

删除操作：删除第 $i(1 \leq i \leq n)$ 个元素，需将第 $i+1$ 到第 n 个元素前移一位。

移动次数 = $n - i$ 。等概率情况下平均移动次数 = $\sum_{i=1}^n (n-i)/n = (n-1)/2$ ，时间复杂度 $O(n)$ 。

【注意】 真题考点：顺序表删除第 i 个元素的平均移动次数为 $(n-1)/2$ （注意是 n 不是 $n+1$ ），插入的平均移动次数为 $n/2$ 。两者分母不同：插入概率分母 $n+1$ ，删除分母 n 。

2.3 单链表（链式存储）

单链表结点包含**数据域**data和**指针域**next。头指针指向第一个结点，尾结点的next为NULL。为操作方便，常在第一个结点前增加一个**头结点**，其数据域不存信息。

```
typedef struct LNode {
    ElemType data;
    struct LNode *next;
} LNode, *LinkList;
```

头插法建表（生成逆序链表）：

```
LinkList CreateList_Head(LinkList L, int n) {
    L = (LinkList)malloc(sizeof(LNode)); // 头结点
    L->next = NULL;
    for (i = 0; i < n; i++) {
        p = (LinkList)malloc(sizeof(LNode));
        scanf(&p->data);
        p->next = L->next; // 新结点指向原首结点
        L->next = p; // 头结点指向新结点
    }
    return L;
}
```

在第i个位置插入结点s（核心两步）：

```
// 前提：已找到第i-1个结点p
s->next = p->next; // ① s指向原第i个结点
p->next = s; // ② p指向s
```

删除第i个结点：

```
// 前提：已找到第i-1个结点p, q = p->next为待删结点
p->next = q->next; // 跳过q
free(q); // 释放空间
```

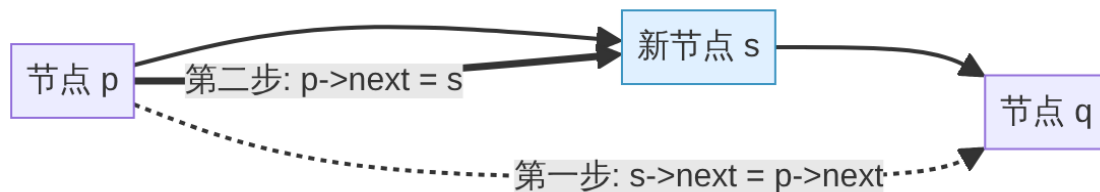


图2-1 单链表插入操作示意图

2.4 循环链表与双向链表

循环链表：尾结点的next指向头结点（或首结点），整个链表形成环。判表空条件：L->next == L（带尾指针rear的循环链表：rear->next->next == rear->next）。

双向链表：每个结点含prior和next两个指针域，可双向遍历。结点定义：

```
typedef struct DuLNode {
    ElemType data;
    struct DuLNode *prior, *next;
} DuLNode, *DuLinkList;
```

双向链表插入结点s到p之后（4步指针操作，顺序重要）：

```
s->prior = p; // ① s的prior指向p
s->next = p->next; // ② s的next指向p的后继
p->next->prior = s; // ③ 原后继的prior指向s
p->next = s; // ④ p的next指向s
```

双向链表删除结点p（2步）：

```
p->prior->next = p->next; // 前驱的next指向p的后继
p->next->prior = p->prior; // 后继的prior指向p的前驱
free(p);
```

2.5 顺序表与链表对比

比较项	顺序表	链表
存储方式	连续存储	任意位置，指针链接
存取方式	随机存取 O(1)	顺序存取 O(n)

比较项	顺序表	链表
插入删除	需移动元素 $O(n)$	修改指针 $O(1)$ （已知位置）
空间开销	无额外指针	每个结点需额外指针域
存储密度	高（ $=1$ ）	低（ <1 ）
适用场景	频繁查询、少增删	频繁增删、少查询

第3章 栈和队列

3.1 栈的定义与操作

栈 (Stack) 是仅允许在表的一端进行插入和删除的线性表。允许操作的一端称为**栈顶 (Top)**，另一端称为**栈底 (Bottom)**。栈的特性是**后进先出 (LIFO, Last In First Out)**。基本操作：Push（入栈）、Pop（出栈）、GetTop（取栈顶）、StackEmpty（判空）。

3.2 顺序栈

顺序栈用一组连续存储单元存放栈元素，附设指针top指示栈顶位置。通常top=-1表示空栈；入栈时先top+1再存元素；出栈时先取元素再top-1。

```
typedef struct {
    ElemType *base; // 栈底指针
    int top; // 栈顶指针(数组下标)
    int stacksize; // 栈容量
} SqStack;

// 入栈
Status Push(SqStack *S, ElemType e) {
    if (S->top == S->stacksize - 1) return ERROR; // 栈满
    S->top++;
    S->base[S->top] = e;
    return OK;
}

// 出栈
Status Pop(SqStack *S, ElemType *e) {
    if (S->top == -1) return ERROR; // 栈空
    *e = S->base[S->top];
    S->top--;
    return OK;
}
```

3.3 栈的应用与卡特兰数

栈的典型应用包括：数制转换、括号匹配、表达式求值（中缀转后缀）、递归调用、迷宫求解等。其中最重要的考点是**出栈序列计数**。

【提示】卡特兰数公式：n个不同元素依次进栈，合法的出栈序列个数为 $C_n = \frac{1}{(n+1)} \times C(2n, n) = \frac{(2n)!}{((n+1)! \times n!)}$ 。常用值： $C_1=1, C_2=2, C_3=5, C_4=14, C_5=42$ 。真题：n=4时，出栈序列有14种。

出栈序列合法性判定：对任一进栈顺序1,2,...,n，若出栈序列为 $p_1, p_2, \dots, p_n$ ，则序列中不存在"3-1-2"模式（即不存在 $i < j < k$ 使 $p_k < p_i < p_j$ ）。

3.4 队列的定义与操作

队列（Queue）是允许在一端（队尾rear）插入、另一端（队头front）删除的线性表。队列特性是先进先出（FIFO, First In First Out）。基本操作：Enqueue（入队）、Dequeue（出队）、QueueEmpty（判空）。

3.5 循环队列

为解决顺序队列的"假溢出"问题，将队列存储空间视为首尾相连的环，称为循环队列。设最大容量MAXSIZE，front指向队头元素，rear指向队尾元素的下一位置：

【提示】循环队列判定（核心公式）：

判空条件：Q.front == Q.rear

判满条件（牺牲一个空间）： $(Q.rear + 1) \% MAXSIZE == Q.front$

队列长度： $(Q.rear - Q.front + MAXSIZE) \% MAXSIZE$

入队：Q.rear = (Q.rear + 1) % MAXSIZE

出队：Q.front = (Q.front + 1) % MAXSIZE

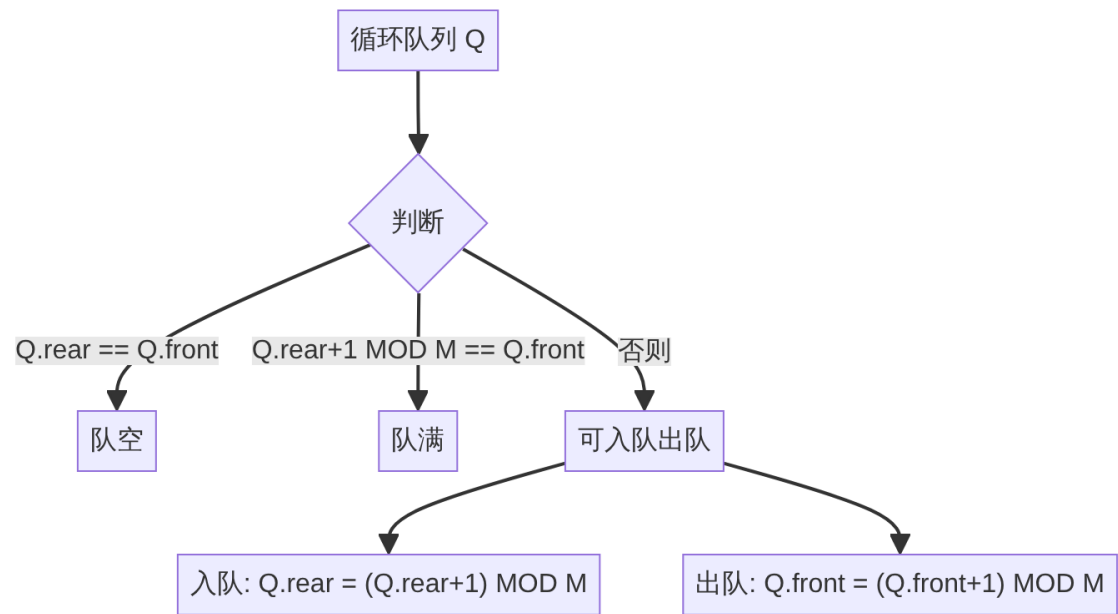


图3-1 循环队列操作示意图

3.6 链队列

链队列用带头结点的单链表实现，附设队头指针front和队尾指针rear，front指向头结点，rear指向尾结点。空队列时front和rear都指向头结点。

【提示】 栈与队列的本质：都是操作受限的线性表。栈限制为单端操作（LIFO），队列限制为双端各管一头（FIFO）。考点：栈适合"回溯/递归/撤销"场景；队列适合"公平调度/缓冲/广度遍历"场景。

第4章 串

4.1 串的基本概念

串 (String) 是零个或多个字符组成的有限序列，记作 $S = "a_1a_2...a_n"$ 。串中所含字符个数n称为**串长**。长度为0的串称为**空串**（记作""）；仅由空格组成的串称为**空格串**（如" "），其长度为空格数。**空串与空格串不同**。

串中任意连续字符组成的子序列称为**子串**，包含子串的串称为**主串**。子串在主串中首次出现时，其首字符在主串中的位序称为子串在主串中的**位置**。两个串相等的充要条件是长度相等且对应位置字符相同。

4.2 串的存储结构

存储方式	说明	特点
定长顺序存储	预分配固定大小数组	存储密度高，可能截断
堆分配存储	动态分配所需空间	灵活，不截断，需管理内存
块链存储	每个结点存多个字符	存储密度可调，操作复杂

4.3 朴素模式匹配（BF算法）

模式匹配：在主串S中查找子串T（模式串）的过程。**BF (Brute-Force)** 算法思想：从主串第pos个字符开始，与模式串首字符对齐比较；若匹配失败，主串指针回退到本次匹配起点+1，模式串指针回退到开头，重新比较。

```
int Index_BF(SSString S, SString T, int pos) {
    int i = pos, j = 1;
    while (i <= S.length && j <= T.length) {
        if (S[i] == T[j]) { i++; j++; } // 匹配则同进
        else { i = i - j + 2; j = 1; } // 回溯
    }
    if (j > T.length) return i - T.length; // 匹配成功
    return 0; // 失败
}
```

【提示】 BF算法时间复杂度：**最好 $O(n+m)$** （一次匹配成功）；**最坏 $O(n \times m)$** （每次匹配到最后一位才失败，如 $S="aaaaaab"$, $T="aaab"$ ）。真题填空：n为主串长m为模式串长，BF时间复杂度为 $O(n \times m)$ 。

4.4 KMP算法

KMP算法利用模式串自身信息，主串指针不回溯，仅模式串指针根据next数组跳转。关键概念：**next[j]**表示模式串第j位失配时，模式串应跳到的下一比较位置。

next数组定义（严蔚敏版）：

next[j] = 模式串第1到j-1位字符组成的子串中，"最长相等前后缀长度 + 1"。特殊地，next[1] = 0。

【提示】 next数组计算步骤（严蔚敏版）：

- ① next[1] = 0;
- ② next[2] = 1（单字符无前后缀）；
- ③ 对j≥3，看T[1..j-1]的最长相等前后缀长度k，next[j] = k + 1;
- ④ 若无相等前后缀，next[j] = 1。

例（真题）：模式串 "ABAABCA" 的next数组计算过程：

j	1	2	3	4	5	6	7
T[j]	A	B	A	A	B	C	A
T[1..j-1]	空	A	AB	ABA	ABAA	ABAAB	ABAABC
最长相等前后缀	—	无	无	A	A	AB	无
长度k	—	0	0	1	1	2	0
next[j]=k+1	0	1	1	2	2	3	1

故 next 数组为 **0,1,1,2,2,3,1**，对应答案A。

4.5 KMP主算法

```
int Index_KMP(SString S, SString T, int pos) {
    int i = pos, j = 1;
    while (i <= S.length && j <= T.length) {
        if (j == 0 || S[i] == T[j]) { i++; j++; }
        else j = next[j]; // 模式串右移，主串不回溯
    }
    if (j > T.length) return i - T.length;
    return 0;
}
```

nextval数组优化：当T[j] == T[next[j]]时，next[j]的跳转无意义，应直接取nextval[j] = nextval[next[j]]。这能减少不必要的比较。

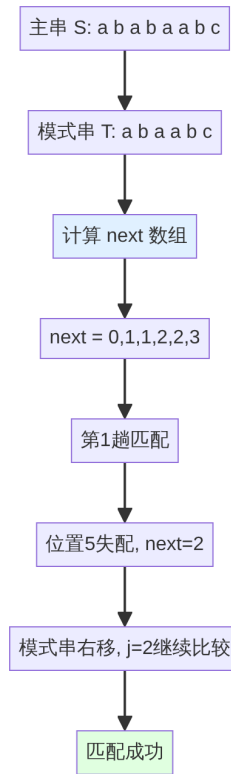


图4-1 KMP算法匹配过程示意

第5章 数组和广义表

5.1 数组的逻辑结构与存储

数组是 $n(n \geq 1)$ 个相同类型数据元素的有序集合，每个元素受 n 个线性关系的约束。数组的存储结构采用**顺序存储**（一维数组物理连续）。多维数组映射到一维有两种方式：**行主序**（以行优先）和**列主序**（以列优先）。

二维数组 $A[m][n]$ 按行主序存储，设每个元素占 c 个存储单元， $LOC(a_{11})$ 为基地址，则：
 $LOC(a[i][j]) = LOC(a_{11}) + ((i-1) \times n + (j-1)) \times c$ （ i, j 从1开始计）。按列主序： $LOC(a[i][j]) = LOC(a_{11}) + ((j-1) \times m + (i-1)) \times c$ 。

5.2 特殊矩阵的压缩存储

为节省空间，对特殊矩阵只存储非零元素，将其映射到一维数组。

矩阵类型	元素特点	存储公式（下标从1计）
对称矩阵	$a[i][j] = a[j][i]$	$k = i*(i-1)/2 + j - 1$ （ $i \geq j$ ，存下三角）
三角矩阵	上三角或下三角全为常数 c	同对称矩阵+1位存常数 c
对角矩阵	仅主对角线及邻近带状区域非零	$k = 3*(i-1) - 1 + (j-i+1)$ （三对角）

【提示】 对称矩阵压缩（真题填空）： $n \times n$ 对称矩阵最少需存储 $n(n+1)/2$ 个元素。理由：上三角（或下三角）+ 对角线共 $1+2+\dots+n = n(n+1)/2$ 个元素。

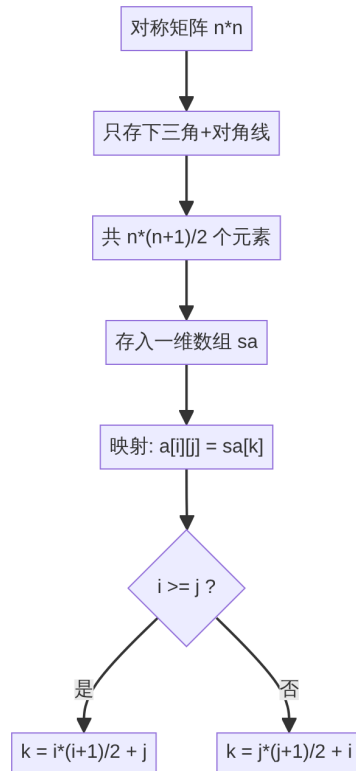


图5-1 对称矩阵压缩存储映射

5.3 稀疏矩阵

稀疏矩阵：非零元素个数远少于矩阵元素总数的矩阵。常用存储方式：**三元组顺序表**（行、列、值）和**十字链表**。

```

typedef struct {
    int i, j; // 行下标, 列下标
    ElemType e; // 元素值
} Triple;

typedef struct {
    Triple data[MAX+1]; // data[0]不用, 从1开始
    int mu, nu, tu; // 行数, 列数, 非零元个数
} TSMatrix;
    
```

稀疏矩阵转置算法：核心是将 (i,j,e) 变为 (j,i,e) ，并按新行序排列。

5.4 广义表

广义表 (Generalized List) 是 $n(n \geq 0)$ 个元素的有限序列，其中每个元素可以是**原子**（不可分割的单元）或**子表**（另一个广义表）。记作 $LS = (a_1, a_2, \dots, a_n)$ ， n 为表长。

基本概念：

- **表头 (Head)**：广义表的第一个元素（可以是原子或子表）
- **表尾 (Tail)**：除去第一个元素后，**剩余元素组成的新表**（必为子表，需加括号）
- **长度**：最外层元素个数
- **深度**：广义表中括号嵌套的最大层数

【注意】表头表尾运算规则（真题考点）：
例 $C = (a, (b, c, d))$
表头 $\text{head}(C) = a$ （单个原子，不加括号）
表尾 $\text{tail}(C) = ((b, c, d))$ （剩余元素组成新表，必须加外层括号）
常见错误：把表尾写成 (b, c, d) 漏掉外层括号。

深度计算例： $L = (a, (b, c, (d, e)))$ ，深度=3（三层括号嵌套）。长度=2（最外层2个元素：a和 $(b, c, (d, e))$ ）。

第6章 树和二叉树

6.1 树的定义与术语

树 (Tree) 是 $n(n \geq 0)$ 个结点的有限集合。 $n=0$ 时为**空树**； $n>0$ 时：①有且仅有一个特定的称为**根 (Root)** 的结点；②其余结点可分为 $m(m \geq 0)$ 个互不相交的有限集合 $T_1, T_2, \dots, T_m$ ，每个集合又是一棵树，称为根的**子树**。

基本术语：

- **度 (Degree)**：结点拥有的子树数；树的度是树中结点的最大度
- **叶子 (Leaf)**：度为0的结点；**分支结点**：度>0的结点
- **层次 (Level)**：根为第1层，其子结点为第2层，依此类推
- **深度 (Depth)**：树中结点的最大层次
- **祖先/子孙**：从根到某结点路径上所有结点是其祖先；某结点子树所有结点是其子孙

6.2 二叉树定义与五大性质

二叉树 (Binary Tree) 是 $n(n \geq 0)$ 个结点的有限集合，或为空集，或由一个根结点和两棵互不相交的、分别称为根的左子树和右子树的二叉树组成。**二叉树不是树的特殊情况**（二叉树区分左右子树，树不区分）。

性质	内容	说明
性质1	第 <i>i</i> 层最多有 $2^{(i-1)}$ 个结点 ($i \geq 1$)	每层结点上限
性质2	深度为 <i>k</i> 的二叉树最多有 $2^k - 1$ 个结点 ($k \geq 1$)	满二叉树达此上限
性质3	$n_0 = n_2 + 1$	叶子数 = 度2结点数 + 1
性质4	<i>n</i> 个结点完全二叉树深度为 $\lceil \log_2 n \rceil + 1$	深度公式

性质	内容	说明
性质5	n个结点完全二叉树，结点i的双亲为 $\lfloor i/2 \rfloor$ ，左孩子 $2i$ ，右孩子 $2i+1$	顺序存储寻址

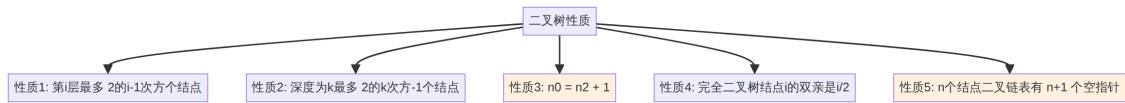


图6-1 二叉树五大性质总结

【提示】 真题考点（必背）：

- ① 深度为k的满二叉树有 $2^k - 1$ 个结点；
- ② n个结点的二叉链表有 **n+1** 个空指针域（推导：2n个指针域，n-1个非空（每个非根结点被指一次），剩 $2n - (n-1) = n+1$ ）；
- ③ $n_0 = n_2 + 1$ ：用于已知部分结点数求其他。

6.3 二叉树遍历

遍历是按某条搜索路径访问每个结点且仅访问一次。四种遍历方式：

遍历方式	访问顺序	递归定义
先序（DLR）	根→左→右	访问根; 先序左子树; 先序右子树
中序（LDR）	左→根→右	中序左子树; 访问根; 中序右子树
后序（LRD）	左→右→根	后序左子树; 后序右子树; 访问根
层序	逐层从左到右	借助队列实现

```
// 先序遍历递归实现
void PreOrder(BiTree T) {
    if (T != NULL) {
        visit(T); // 访问根
        PreOrder(T->lchild); // 递归左子树
        PreOrder(T->rchild); // 递归右子树
    }
}

// 中序遍历(只需调换visit的位置)
void InOrder(BiTree T) {
    if (T != NULL) {
        InOrder(T->lchild);
        visit(T);
        InOrder(T->rchild);
    }
}
```

【提示】 由遍历序列恢复二叉树：必须知道**中序**+（先序或后序）**任一种**。①先序+中序：先序首元素为根，在中序中定位根，左侧为左子树，右侧为右子树，递归处理。②后序+中序：后序末元素为根，其余同理。

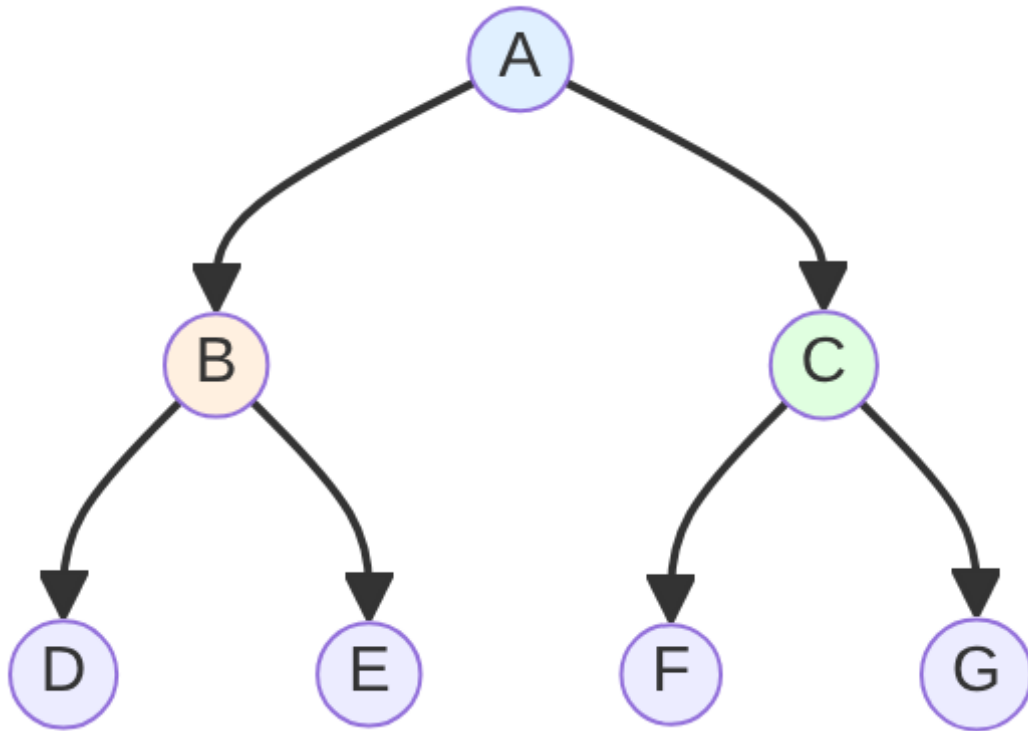


图6-2 二叉树遍历示例

6.4 线索二叉树

n 个结点的二叉链表有 $n+1$ 个空指针域，可用于存放**线索**（指向前驱/后继的指针）。结点结构增加两个标志域 ltag 和 rtag：

- ltag=0: lchild指向左孩子；ltag=1: lchild指向前驱（线索）
- rtag=0: rchild指向右孩子；rtag=1: rchild指向后继（线索）

中序线索二叉树的中序后继查找函数 nextnode（真题原题）：


```

BiThrNode *nextnode(BiThrNode *p) {
    if (p->rtag == 1) {
        return p->rchild; // 右指针是线索，直接返回
    } else {
        p = p->rchild; // 进入右子树
        while (p->ltag == 0) // 找右子树的最左下结点
            p = p->lchild;
        return p;
    }
}

// 中序遍历线索二叉树(不用栈,不用递归)
void inorder_thr(BiThrTree T) {
    if (T == NULL) return;
    BiThrNode *p = T;
    while (p->ltag == 0) p = p->lchild; // 找最左下
    while (p != NULL) {
        visit(p);
        p = nextnode(p);
    }
}

```

6.5 树、森林与二叉树的转换

孩子兄弟表示法：每个结点设两个指针，分别指向第一个孩子和下一个亲兄弟。通过这种表示法，任何树都可转为二叉树。

【提示】转换规则：

树 → 二叉树：左孩子 = 原树第一个孩子；右孩子 = 原树下一个兄弟。根结点无兄弟，故二叉树根的右子树始终为空。

森林 → 二叉树：先按上述规则把每棵树转为二叉树（根的右子树为空），然后依次把第2棵作为第1棵根的右子树，第3棵作为第2棵根的右子树，依此类推。

遍历对应：树的先根遍历 = 二叉树先序；树的后根遍历 = 二叉树中序。

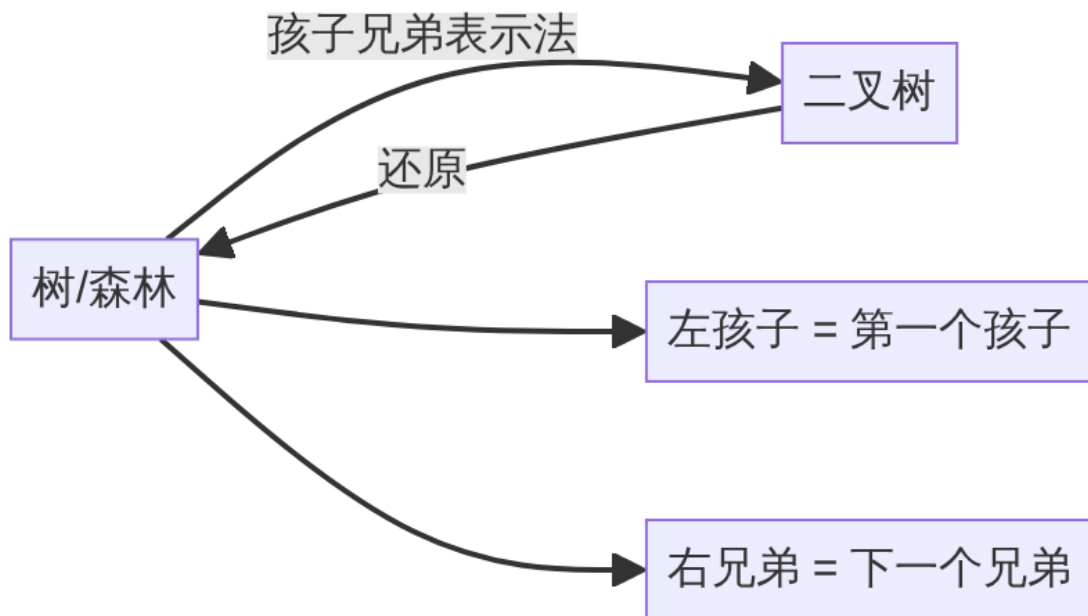


图6-3 树/森林与二叉树转换规则

6.6 哈夫曼树与哈夫曼编码

路径长度：从树根到某结点路径上的分支数。**带权路径长度（WPL）：**树的所有叶子结点的路径长度 $\times$ 权值之和。**哈夫曼树（最优二叉树）：**给定 n 个权值作为叶子，构造一棵二叉树，使WPL最小。

哈夫曼树构造算法：

- ① 将 n 个权值看作 n 棵只有根的树，构成森林 F
- ② 在 F 中选两棵根权值最小的树合并为新树，新树根权=两子树根权之和
- ③ 从 F 中删除这两棵树，加入新树
- ④ 重复②③直到 F 中只剩一棵树，即为哈夫曼树

【提示】哈夫曼树性质：① 没有度为1的结点（每次合并两棵子树，故度为0或2）；② n 个叶子结点的哈夫曼树共有 $2n-1$ 个结点（ $n_0=n, n_2=n-1$, 总 $=2n-1$ ）；③ $WPL =$ 所有非叶子结点权值之和（或所有叶子权值 $\times$ 路径长度之和）。

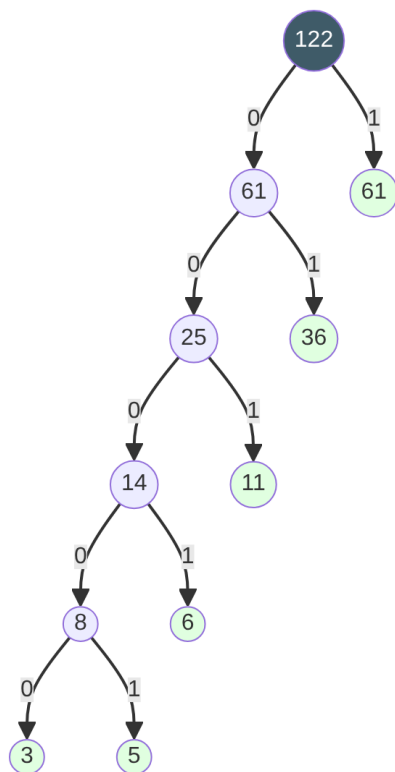


图6-4 哈夫曼树构造示例

哈夫曼编码：将字符频率作为权值构造哈夫曼树，左分支标0右分支标1（或反之），从根到叶子的路径即为该字符的编码。哈夫曼编码是**前缀编码**（任一字符编码都不是另一字符编码的前缀），解码时不会产生歧义。

第7章 图

7.1 图的定义与术语

图（Graph）由顶点集合 V 和边集合 E 组成，记作 $G = (V, E)$ 。 V 是顶点的有穷非空集合， E 是顶点间关系（边）的有穷集合。**无向图**的边是无序对 (v, u) ，**有向图**的边是有序对 $\langle v, u \rangle$ （称为弧）。

重要术语：

- **完全图：**无向图有 $n(n-1)/2$ 条边（任意两顶点间有边）；有向图有 $n(n-1)$ 条弧
- **顶点的度：**与该顶点相关联的边数。有向图分**入度**ID（以该点为弧头）和**出度**OD（以该点为弧尾）
- **路径：**顶点序列 $v_1, v_2, \dots, v_n$ ，其中 $(v_i, v_{i+1}) \in E$ ；路径长度=边数
- **连通图：**无向图中任意两顶点间有路径；**连通分量：**极大连通子图
- **强连通图：**有向图中任意两顶点相互可达；**强连通分量：**极大强连通子图
- **生成树：**连通图的极小连通子图，含 n 个顶点 $n-1$ 条边

7.2 图的存储结构

邻接矩阵：用 $n \times n$ 矩阵A存储， $A[i][j]=1$ 表示 $\langle v_i, v_j \rangle$ 是边（或弧）， $=0$ 表示不是。

【提示】 邻接矩阵特性：① 无向图邻接矩阵**对称**，可压缩存储；② 无向图顶点 v_i 的度 = 第i行(或第i列)元素之和；③ 有向图顶点 v_i 的出度 = 第i行元素之和，入度 = 第i列元素之和；④ 矩阵非零元个数 = 无向图边数2倍 / 有向图弧数；⑤ 适合稠密图，空间 $O(n^2)$ 。

邻接表：为每个顶点建立一个单链表，链表中存放与该顶点相邻接的所有顶点。表头用数组存储，每个表头结点含vertex和firstedge两个域。

【提示】 真题考点：无向图有n个顶点m条边，邻接表有 **2m** 个边结点（每条边在两个顶点的链表中各出现一次）。有向图有m条弧，邻接表有 **m** 个边结点。空间复杂度 $O(n+m)$ 。

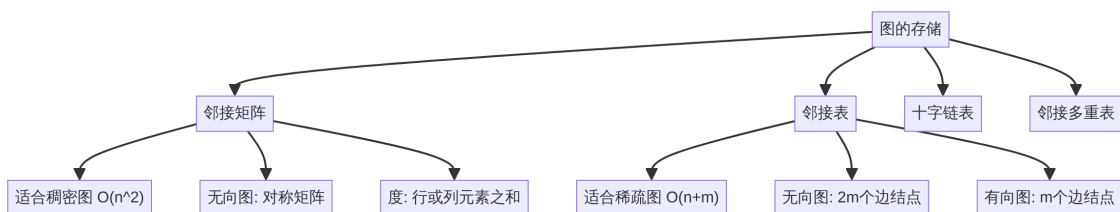


图7-1 图的存储结构对比

7.3 图的遍历

深度优先搜索（DFS）：类似树的先根遍历。从某顶点v出发，访问v；然后依次从v的未被访问的邻接点出发，递归进行DFS。需要**栈**（递归调用栈或显式栈）辅助。

```

bool visited[MAX];
void DFS(Graph G, int v) {
    visit(v);
    visited[v] = true;
    for (w = FirstNeighbor(G, v); w >= 0; w = NextNeighbor(G, v, w))
        if (!visited[w]) DFS(G, w);
}
    
```

广度优先搜索（BFS）：类似树的层序遍历。从某顶点v出发，访问v；然后依次访问v的所有未被访问的邻接点；再按这些邻接点被访问的先后次序，依次访问它们的邻接点。需要**队列**辅助。

```

void BFS(Graph G, int v) {
    Queue Q; InitQueue(Q);
    visit(v); visited[v] = true; EnQueue(Q, v);
    while (!QueueEmpty(Q)) {
        DeQueue(Q, u);
        for (w = FirstNeighbor(G, u); w >= 0; w = NextNeighbor(G, u, w))
            if (!visited[w]) {
                visit(w); visited[w] = true; EnQueue(Q, w);
            }
    }
}
    
```

【提示】 DFS/BFS 时间复杂度：邻接矩阵存储 $O(n^2)$ ，邻接表存储 $O(n+e)$ 。空间复杂度：均 $O(n)$ （visited数组 + 栈或队列）。

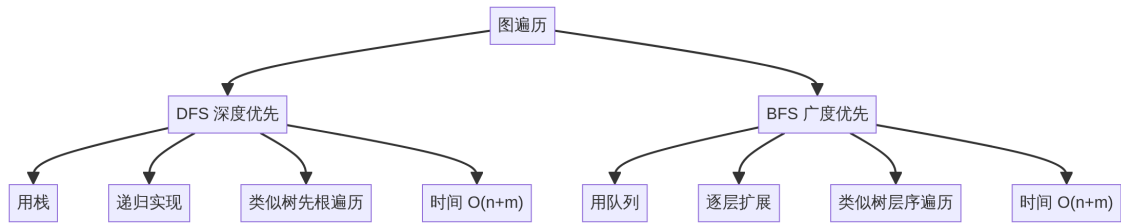


图7-2 DFS与BFS对比

7.4 最小生成树 (MST)

最小生成树：连通带权图中，所有生成树中权值之和最小的那棵。两种经典算法：

Prim算法（普里姆）：从任一顶点开始，逐步将顶点加入MST。每次选横跨已选顶点集U和未选顶点集V-U的、权值最小的边，对应的顶点加入U。适合**稠密图**，时间 $O(n^2)$ 。

Kruskal算法（克鲁斯卡尔）：将所有边按权值升序排序，依次考察每条边；若该边连接的两个顶点不在同一连通分量（加入后不形成回路），则加入MST。适合**稀疏图**，时间 $O(e \log e)$ 。需借助**并查集**判断回路。

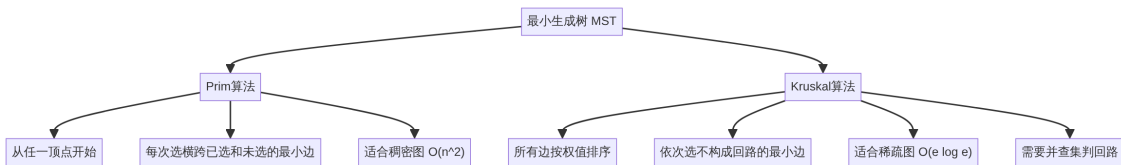


图7-3 Prim与Kruskal算法对比

7.5 最短路径

Dijkstra算法（迪杰斯特拉）：求单源最短路径（从一个顶点到其他所有顶点）。基本思想：将顶点集V分为已求出最短路径的集合S和未求出的集合T，每次从T中选距离源点最近的顶点u加入S，然后以u为中间点更新T中顶点的距离。

【注意】Dijkstra算法限制：不适用**负权边**的图（贪心策略在负权下不成立）。时间复杂度： $O(n^2)$ 。能处理有向图和无向图，但要求权值非负。

Dijkstra算法执行步骤（以真题为例：v1→v2:8, v1→v3:100, v2→v3:60, v2→v4:80, v3→v4:5）：

步骤	选中顶点	dist[v2]	dist[v3]	dist[v4]	已访问集合S
初始	—	∞	∞	∞	{v1}
1	v1(0)	8	100	∞	{v1,v2}
2	v2(8)	8	min(100,8+60)=68	8+80=88	{v1,v2,v3}
3	v3(68)	8	68	min(88,68+5)=73	{v1,v2,v3,v4}

最终结果：v1→v2长度8；v1→v3长度68（路径v1→v2→v3）；v1→v4长度73（路径v1→v2→v3→v4）。

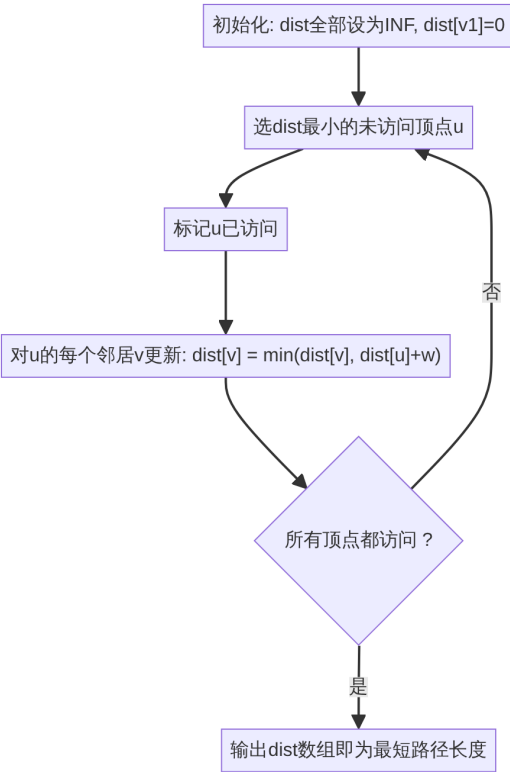


图7-4 Dijkstra算法流程

7.6 拓扑排序与关键路径（概念）

AOV网（Activity On Vertex）：以顶点表示活动、弧表示活动先后关系的有向无环图（DAG）。**拓扑排序**：对AOV网进行排序，使所有活动的前驱活动都排在其前面。算法：每次选入度为0的顶点输出并删除其所有出边，重复直到所有顶点输出（若剩余顶点入度均不为0则存在环）。

AOE网（Activity On Edge）：以边表示活动、顶点表示事件的有向无环图。**关键路径**：从源点到汇点路径长度最长的路径，其上活动称为关键活动。关键活动的延误会影响整个工程周期。

第9章 查找

9.1 查找的基本概念

查找表是由同一类型数据元素构成的集合。**关键字**是数据元素中某个数据项的值，用于标识该元素。若能唯一标识一个元素，称为主关键字；否则为次关键字。**静态查找表**：仅做查找和读取操作，不增删；**动态查找表**：查找时同时进行增删。

平均查找长度（ASL）是衡量查找算法效率的主要指标： $ASL = \sum (P_i \times C_i)$ ，其中 P_i 为查找第 i 个元素的概率， C_i 为查找第 i 个元素所需的比较次数。

9.2 静态查找

顺序查找：从表的一端开始，逐个比较关键字。为提高效率，常设**监视哨**（将待查值存入ST.elem[0]）：

```
int Search_Seq(SSTable ST, KeyType key) {
    ST.elem[0].key = key; // 监视哨, 免去越界判断
    for (i = ST.length; ST.elem[i].key != key; i--);
    return i; // 返回0表示查找失败
}
```

ASL：等概率下 $ASL = (n+1)/2$ 。时间复杂度 $O(n)$ 。

折半查找（二分查找）：要求**顺序存储且关键字有序**。每次比较中间元素，若相等则成功；若待查值小于中间元素，则在左半区继续查找；否则在右半区继续。

```
int Search_Bin(SSTable ST, KeyType key) {
    int low = 1, high = ST.length, mid;
    while (low <= high) {
        mid = (low + high) / 2;
        if (ST.elem[mid].key == key) return mid;
        else if (key < ST.elem[mid].key) high = mid - 1;
        else low = mid + 1;
    }
    return 0;
}
```

【提示】折半查找判定树是**平衡二叉树**（或类似），查找过程对应判定树上一条从根到目标结点的路径。 $ASL \approx \log_2(n+1) - 1$ 。时间复杂度 $O(\log n)$ 。**注意：折半查找不适用于链表**（无法随机存取）。

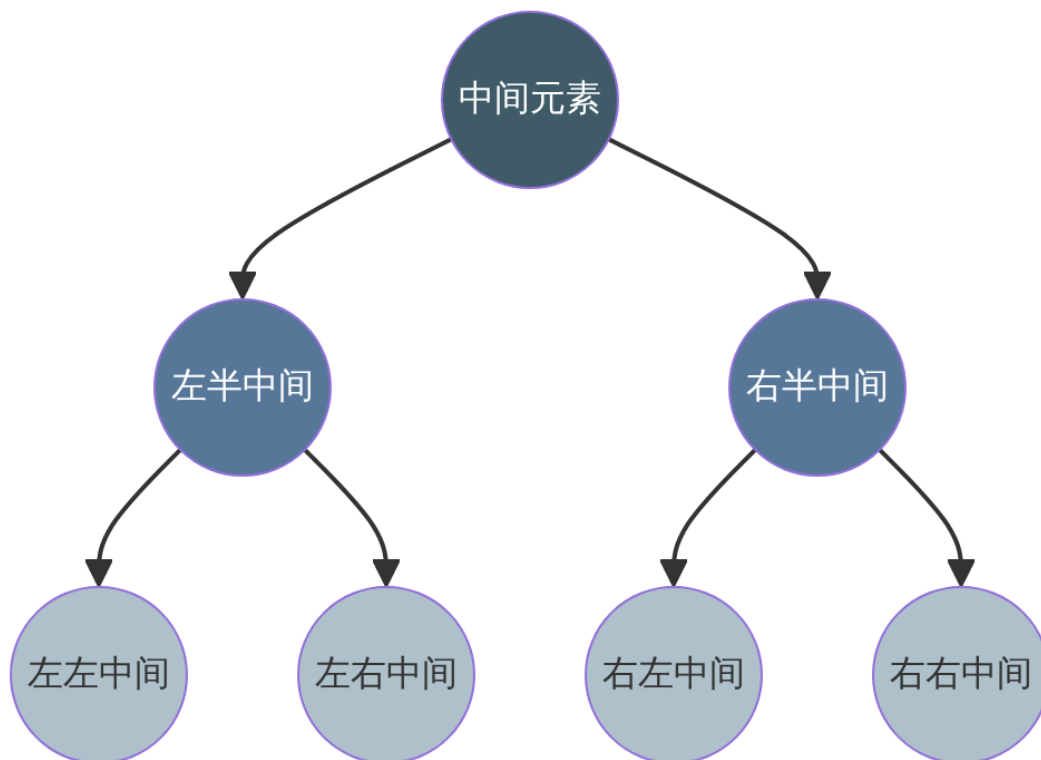


图9-1 折半查找判定树结构

分块查找（索引顺序查找）：将表分若干块，块间有序（第i块最大值<第i+1块最小值），块内无序。先在索引表折半查找定位块，再在块内顺序查找。ASL = L_1 (索引) + L_2 (块内)。适合**分块有序**的静态查找表（真题考点）。

9.3 二叉排序树（BST）

二叉排序树：空树或满足：①左子树所有结点值 < 根值；②右子树所有结点值 > 根值；③左右子树也是二叉排序树。中序遍历BST得到**递增有序序列**。

BST查找算法：

```
BiTree SearchBST(BiTree T, KeyType key) {
    if (T == NULL || T->data.key == key) return T;
    else if (key < T->data.key) return SearchBST(T->lchild, key);
    else return SearchBST(T->rchild, key);
}
```

BST插入：新结点必成为叶子结点。从根比较，小于向左、大于向右，直到空位置插入。

BST删除：分三种情况：

- ① 删除叶子：直接删
- ② 删除单分支结点：用其唯一孩子替代
- ③ 删除双分支结点：用其中序前驱（左子树最右下）或中序后继（右子树最左下）替代，再递归删除前驱/后继

9.4 平衡二叉树（AVL）

平衡二叉树（AVL树）：任意结点的左右子树高度差（**平衡因子BF**）的绝对值不超过1的BST。当插入或删除导致失衡时，通过**旋转调整**。

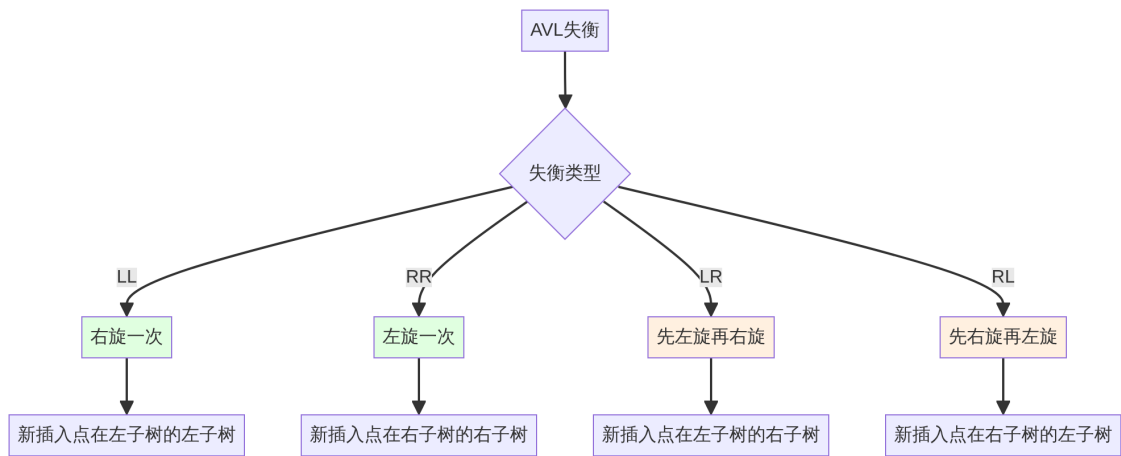


图9-2 AVL四种旋转类型

失衡类型	判定	调整方法	说明
LL型	在左子树的左子树插入	右旋一次	A的左子B上升，A成为B的右子
RR型	在右子树的右子树插入	左旋一次	A的右子B上升，A成为B的左子

失衡类型	判定	调整方法	说明
LR型	在左子树的右子树插入	先左旋再右旋	先对A的左子左旋(变LL)，再对A右旋
RL型	在右子树的左子树插入	先右旋再左旋	先对A的右子右旋(变RR)，再对A左旋

【提示】AVL考点：n个结点的AVL树高度为 $O(\log n)$ ，查找/插入/删除均为 $O(\log n)$ 。与普通BST相比，AVL避免了最坏情况（链式BST退化为 $O(n)$ ）。

9.5 哈希表

哈希表 (Hash Table)：通过哈希函数 $H(\text{key})$ 直接计算关键字的存储地址，理想情况下查找/插入/删除均为 $O(1)$ 。

哈希函数构造方法：

- **直接定址法**： $H(\text{key}) = a \times \text{key} + b$ ，适合关键字分布连续
- **除留余数法**： $H(\text{key}) = \text{key} \text{ MOD } p$ （ p 取不大于表长的最大质数），最常用
- **数字分析法**：取关键字中分布均匀的若干位
- **平方取中法**：取关键字平方的中间几位

冲突解决方法：

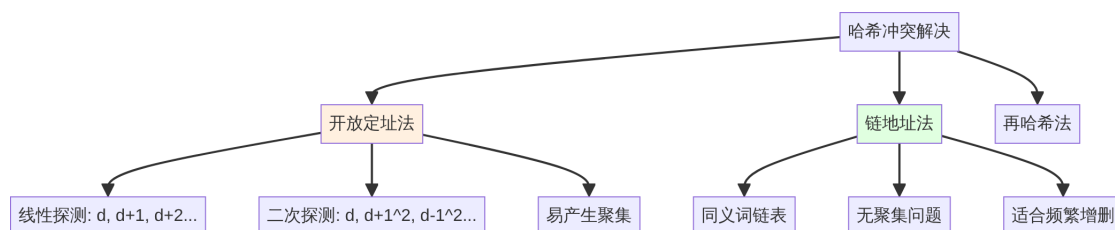


图9-3 哈希冲突解决方法对比

开放定址法：冲突时按某种规则探测下一个空位置。线性探测： $H_i = (H(\text{key}) + i) \text{ MOD } m$ ；二次探测： $H_i = (H(\text{key}) + i^2) \text{ MOD } m$ 。

链地址法 (拉链法)：将所有哈希地址相同的关键字存储在单一单链表中。无"聚集"问题，适合频繁增删。

【注意】ASL计算（真题考点）：给定关键字集合和哈希函数，构造哈希表后：

ASL成功 = (各关键字的探测次数之和) / 关键字个数

ASL失败 = (各位置的探测次数之和) / 表长

注意：线性探测法失败时，从该位置开始直到空位置的探测次数（包括空位置本身）。

真题示例：关键字(25,10,8,27,32,68,5,66,69)， $H(k)=k\%11$ ，表长12：

线性探测法哈希表（带下划线的为冲突后探测的位置）：

位置	0	1	2	3	4	5	6	7	8	9	10	11
关键字	66	空	68	25	69	27	5	空	8	空	10	32
探测次数	1	—	1	1	2	1	2	—	1	—	1	2

$$ASL_{成功} = (1+1+1+2+1+2+1+1+2)/9 = 12/9 = 4/3$$

第10章 排序

10.1 排序的基本概念

排序 (Sorting)：将一组记录按关键字递增（或递减）的次序排列起来。**稳定性**：若关键字相同的两个记录在排序前后相对位置不变，则称排序方法是稳定的，否则不稳定。**内部排序**：整个排序过程在内存中进行；**外部排序**：数据量大需借助外存。

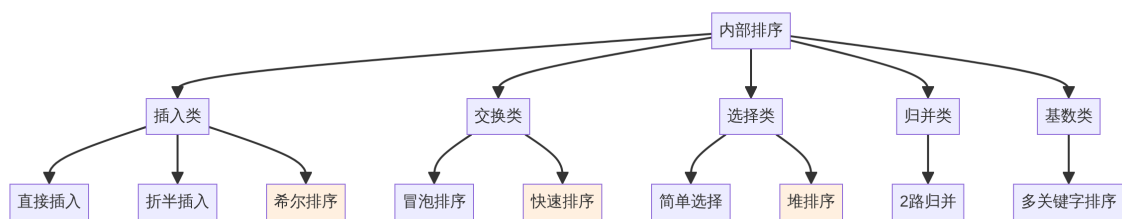


图10-1 内部排序算法分类

10.2 插入排序

直接插入排序：将第*i*个记录插入到前*i*-1个已排好序的子序列中适当位置。从*i*=2开始，将*R*[*i*]与*R*[*i*-1],*R*[*i*-2],...依次比较，找到插入位置。

```

void InsertSort(int A[], int n) {
    for (i = 2; i <= n; i++) {
        if (A[i] < A[i-1]) {
            A[0] = A[i]; // 哨兵
            for (j = i-1; A[j] > A[0]; j--)
                A[j+1] = A[j]; // 后移
            A[j+1] = A[0]; // 插入
        }
    }
}
  
```

直接插入排序：最好（已有序） $O(n)$ ，最坏（逆序） $O(n^2)$ ，平均 $O(n^2)$ 。**稳定**。适合基本有序或*n*小的场景。

折半插入排序：在直接插入基础上，用折半查找定位插入位置，减少比较次数。比较次数 $O(n \log n)$ ，但移动次数仍为 $O(n^2)$ ，总复杂度仍为 $O(n^2)$ 。**稳定**。

希尔排序 (Shell Sort)：分组插入排序。选一个增量序列 $d_1 > d_2 > \dots > d_t = 1$ ，对每个 d_i 将整个序列分成 d_i 组，组内直接插入排序。

【提示】 真题示例：序列(21,25,49,5*,16,8)，初始gap=3，分组：(21,5*)→(5*,21)，(25,16)→(16,25)，(49,8)→(8,49)。一趟结果：(5*, 16, 8, 21, 25, 49)。希尔排序**不稳定**（相同关键字可能分到不同组）。

10.3 交换排序

冒泡排序：相邻元素两两比较，逆序则交换。每趟将最大元素"冒泡"到末尾。n-1趟后有序。优化：若某趟无交换则提前结束。

快速排序：选一个基准（pivot，通常首元素），通过一趟划分将序列分为两部分：左侧均 $\leq$ pivot，右侧均 $\geq$ pivot；然后递归对两部分排序。

```
int Partition(int A[], int low, int high) {
    int pivot = A[low];
    while (low < high) {
        while (low < high && A[high] >= pivot) high--;
        A[low] = A[high];
        while (low < high && A[low] <= pivot) low++;
        A[high] = A[low];
    }
    A[low] = pivot;
    return low;
}
```

```
void QuickSort(int A[], int low, int high) {
    if (low < high) {
        int pivotpos = Partition(A, low, high);
        QuickSort(A, low, pivotpos - 1);
        QuickSort(A, pivotpos + 1, high);
    }
}
```

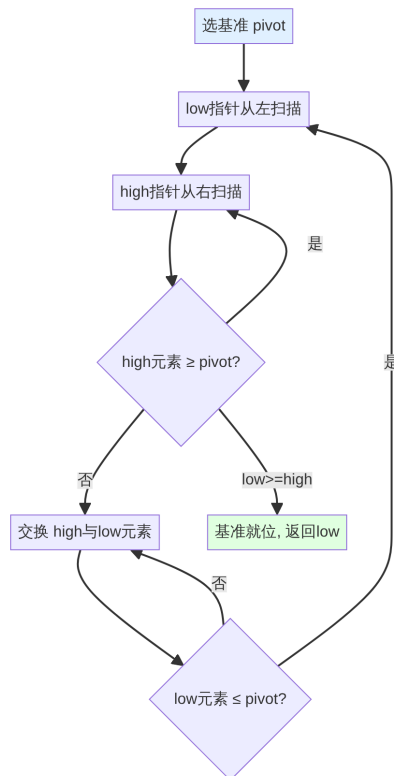


图10-2 快速排序Partition过程

【注意】快排复杂度（真题考点）：

最好/平均时间： $O(n \log n)$ ，空间 $O(\log n)$ （递归栈）

最坏时间： $O(n^2)$ ，空间 $O(n)$ （序列已有序且每次选首元素为基准，退化为冒泡）

最坏情况下比较次数和移动次数均为 $O(n^2)$ 。快排不稳定。

10.4 选择排序

简单选择排序：第*i*趟从后*n-i+1*个元素中选最小元素，与第*i*个元素交换。比较次数固定为 $n(n-1)/2$ ，交换次数最多 $n-1$ 。**不稳定**。

堆排序：利用堆的性质进行排序。**大顶堆**：每个结点的值 $\geq$ 其孩子结点的值（根最大）。**小顶堆**：每个结点的值 $\leq$ 其孩子结点的值（根最小）。

建堆过程：从最后一个非叶子结点（下标 $\lfloor n/2 \rfloor$ ）开始，向前逐个执行**向下调整**（sift）。

```
// 向下调整(以大顶堆为例)
void sift(int A[], int low, int high) {
    int i = low, j = 2 * i; // j为左孩子
    while (j <= high) {
        if (j < high && A[j] < A[j+1]) j++; // j指向较大孩子
        if (A[i] >= A[j]) break; // 已满足堆性质
        swap(A[i], A[j]); // 交换
        i = j; j = 2 * i; // 继续向下
    }
}

// 建堆：从n/2开始向前调整
for (i = n/2; i >= 1; i--) sift(A, i, n);

// 堆排序
void HeapSort(int A[], int n) {
    for (i = n/2; i >= 1; i--) sift(A, i, n); // 建堆
    for (i = n; i > 1; i--) {
        swap(A[1], A[i]); // 交换堆顶和末尾
        sift(A, 1, i-1); // 调整剩余元素
    }
}
```

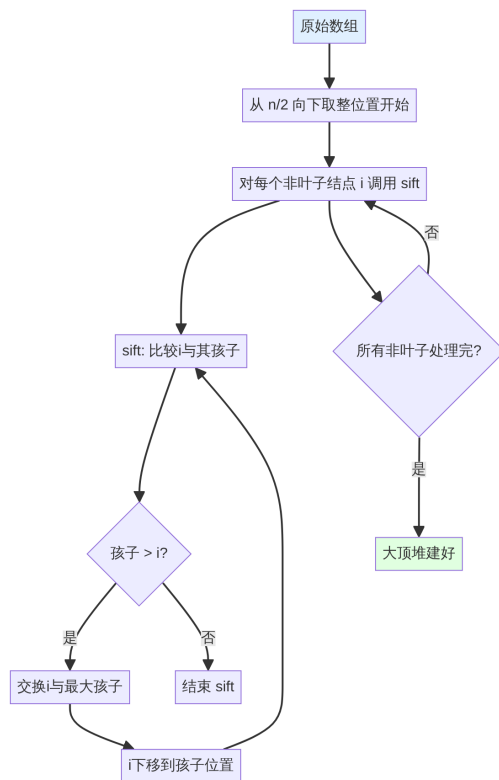


图10-3 堆排序建堆过程

【提示】 真题示例：序列(68, 57, 49, 65, 79, 19, 46, 13)建大顶堆。从 $i=4$ 开始（65的孩子19不调整）， $i=3$ (49的孩子46,13不调整)， $i=2$ (57的孩子65,79，79更大，交换57和79)， $i=1$ (68的孩子79,49，79更大，交换68和79，68的孩子变为65,57，65更大，交换68和65)。最终大顶堆：(79, 68, 49, 65, 57, 19, 46, 13)。

堆排序时间复杂度：建堆 $O(n)$ ，每次调整 $O(\log n)$ ，共 $n-1$ 次，总 $O(n \log n)$ 。**不稳定**。适合大数据量、不需全部排序只求top-k的场景。

10.5 归并排序与基数排序（概念）

归并排序：将相邻有序子序列两两合并，直至整个序列有序。2路归并：每趟将相邻两个有序子序列合并，共 $\lceil \log_2 n \rceil$ 趟。时间 $O(n \log n)$ ，空间 $O(n)$ （需要辅助数组）。**稳定**。

基数排序：借助"分配"和"收集"进行排序。对 d 位关键字，每位进行 n 个分配和 r 个收集（ r 为基数）。时间 $O(d(n+r))$ ，空间 $O(r+n)$ 。**稳定**。适合关键字位数少且取值范围小的场景。

10.6 排序算法总结

算法	最好时间	平均时间	最坏时间	空间	稳定性
直接插入	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
折半插入	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
希尔排序	—	$O(n^{1.3})$	$O(n^2)$	$O(1)$	不稳定
冒泡排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定

算法	最好时间	平均时间	最坏时间	空间	稳定性
快速排序	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	不稳定
简单选择	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定
堆排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	不稳定
2路归并	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	稳定
基数排序	—	$O(d(n+r))$	$O(d(n+r))$	$O(r+n)$	稳定

【提示】稳定性记忆口诀：插冒归基稳（直接插入、冒泡、归并、基数 稳定）；希快选堆不稳（希尔、快排、简单选择、堆 不稳定）。**时间效率：**快排平均最快；堆排最坏也快；归并稳定且快但费空间。

第二部分 公式速记卡与对照表

1. 高频公式速记卡

以下公式为历年考试高频考点，务必熟记：

编号	公式	用途
F1	$C_n = (1/(n+1)) \times C(2n,n) = (2n)!/((n+1)!n!)$	n个元素出栈序列数（卡特兰数）
F2	顺序表插入平均移动 = $n/2$ ，删除 = $(n-1)/2$	顺序表操作复杂度分析
F3	满二叉树深度k结点数 = $2^k - 1$	满二叉树性质
F4	完全二叉树深度 = $\lfloor \log_2 n \rfloor + 1$	完全二叉树深度公式
F5	n个结点二叉链表空指针数 = $n + 1$	线索化基础
F6	二叉树性质： $n_0 = n_2 + 1$	叶子与度2结点关系
F7	无向图邻接表边结点数 = $2m$	无向图存储空间计算
F8	完全无向图边数 = $n(n-1)/2$ ，完全有向图 = $n(n-1)$	完全图边数
F9	循环队列：判空 $front == rear$ ；判满 $(rear+1) \% M == front$	循环队列操作
F10	对称矩阵压缩存储元素数 = $n(n+1)/2$	特殊矩阵压缩
F11	哈希表ASL成功 = $\sum P_i C_i / n$ ；ASL失败 = $\sum C_i / \text{表长}$	哈希表性能分析
F12	哈夫曼树总结点数 = $2n-1$ （n为叶子数）；WPL = $\sum \text{叶子权值} \times \text{路径长}$	哈夫曼树计算
F13	BST中序遍历得递增序列	BST性质
F14	AVL树高度 = $O(\log_2 n)$	AVL性能保证
F15	快排最坏 $O(n^2)$ ，平均 $O(n \log n)$ ；堆排/归并 $O(n \log n)$	排序复杂度
F16	折半查找ASL $\approx \log_2(n+1) - 1$	折半查找性能
F17	顺序查找ASL = $(n+1)/2$	顺序查找性能
F18	Dijkstra不适用负权；时间 $O(n^2)$	最短路径算法
F19	Prim $O(n^2)$ 适合稠密图；Kruskal $O(e \log e)$ 适合稀疏图	MST算法选择
F20	DFS用栈，BFS用队列；邻接表 $O(n+e)$ ，邻接矩阵 $O(n^2)$	图遍历复杂度

2. 复杂度对照表

2.1 排序算法复杂度

算法	最好	平均	最坏	空间	稳定	备注
直接插入	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	是	基本有序最快
折半插入	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	是	比较次数减半

算法	最好	平均	最坏	空间	稳定	备注
希尔	—	$O(n^{1.3})$	$O(n^2)$	$O(1)$	否	增量序列影响
冒泡	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	是	加标志位优化
快速	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	否	平均最快
简单选择	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	否	比较次数固定
堆排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	否	最坏也快
归并	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	是	稳定且快但费空间
基数	—	$O(d(n+r))$	$O(d(n+r))$	$O(r+n)$	是	关键字位数少时快

2.2 查找算法复杂度

算法	ASL成功	ASL失败	时间复杂度	空间	备注
顺序查找	$(n+1)/2$	$n+1$	$O(n)$	$O(1)$	无序有序均可
折半查找	$\approx \log_2(n+1)-1$	$\approx \log_2 n$	$O(\log n)$	$O(1)$	需有序+顺序存储
分块查找	L_1+L_2	—	$O(\log n + s)$	$O(1)$	需分块有序
BST查找	—	—	$O(\log n) \sim O(n)$	$O(1)$	取决于树形态
AVL查找	—	—	$O(\log n)$	$O(1)$	平衡保证
哈希查找	$\approx 1+\alpha/2$	—	$O(1)$ 平均	$O(m)$	α 为装填因子

2.3 图算法复杂度

算法	邻接矩阵	邻接表	空间	备注
DFS	$O(n^2)$	$O(n+e)$	$O(n)$	用栈
BFS	$O(n^2)$	$O(n+e)$	$O(n)$	用队列
Prim	$O(n^2)$	$O(n^2)$	$O(n)$	适合稠密图
Kruskal	$O(n^2)$	$O(e \log e)$	$O(n+e)$	适合稀疏图，需并查集
Dijkstra	$O(n^2)$	$O((n+e) \log n)$	$O(n)$	不适用负权
Floyd	$O(n^3)$	$O(n^3)$	$O(n^2)$	求多源最短路径
拓扑排序	$O(n^2)$	$O(n+e)$	$O(n)$	DAG专用

3. 易错点速查表

以下为各章高频易错点汇总，考前务必过一遍：

章节	易错点	正确做法/说明
第1章	时间复杂度只看最高阶项，忽略系数和低阶项	$10n^2+100n = O(n^2)$ ，不是 $O(10n^2)$ ； $O(n \log n)$ 与 $O(\log n!)$ 等价
第2章	顺序表删除移动次数分母是n，插入是n+1	删除平均 $(n-1)/2$ ，插入平均 $n/2$ ；混淆是常见错误

章节	易错点	正确做法/说明
第2章	头结点≠头指针：头指针指向首结点（含头结点）	带头结点的链表，第一个数据结点是L->next；判空：L->next==NULL
第3章	循环队列"牺牲一个空间"判满	判满用(rear+1)%M==front，不是rear+1==front（越界）
第3章	卡特兰数适用前提：n个元素 依次 进栈	若允许中途插入，则不适用； $C_4=14$ 是真题原考
第4章	KMP的next数组两种定义	严蔚敏版首位为0（next[1]=0）；通用版首位为-1。本课程用前者
第4章	BF最坏 $O(n \times m)$ ，KMP最坏 $O(n+m)$	KMP主串指针不回溯，但模式串需预处理next数组
第5章	广义表表尾必须加括号	$C=(a,(b,c))$ 的tail是 $((b,c))$ ，不是 (b,c)
第5章	对称矩阵下标公式 $i \geq j$ 时 $k=i(i-1)/2+j-1$ （下标从1计）	注意下标起点：若从0计则公式不同
第6章	二叉树空指针数 $n+1$ 的推导	$2n$ 个指针域， $n-1$ 个非空（每非根结点被指一次），剩 $n+1$ 空
第6章	线索化ltag=1表示lchild是线索（指向前驱），不是孩子	混淆ltag=0/1的含义会找错前驱后继
第6章	哈夫曼树没有度为1的结点	总结点数 $=2n-1$ ；非叶子全为度2
第6章	由先序+中序（或后序+中序）可唯一确定二叉树	仅先序+后序不能唯一确定二叉树
第7章	无向图邻接表边结点 $2m$ ，有向图 m	无向图每边在两端各出现一次
第7章	DFS用栈BFS用队列	记忆：DFS深度→回溯→栈；BFS广度→逐层→队列
第7章	Dijkstra不适用负权	贪心策略在负权下不成立；负权用Bellman-Ford
第7章	完全无向图边数 $n(n-1)/2$	不是 $n^2/2$ ；完全有向图 $n(n-1)$
第9章	折半查找要求顺序存储且有序	链表无法随机存取，不能用折半查找
第9章	AVL旋转方向判断：LL右旋，RR左旋	LL=在左子树的左子树插入→右旋调整；命名规则：插入位置序列
第9章	哈希表ASL成功≠ASL失败	成功：关键字探测和/关键字数；失败：位置探测和/表长
第10章	稳定性判定：插冒归基稳，希快选堆不稳	记忆口诀，必背
第10章	快排最坏 $O(n^2)$ 在序列已有序时	可选中间元素作pivot避免最坏情况
第10章	堆排序建堆起点 $\lfloor n/2 \rfloor$ （最后一个非叶子）	不是从根开始；从后向前调整
第10章	希尔排序不稳定	相同关键字可能分到不同子序列，相对顺序改变

第三部分 分板块模拟题

本部分按章节分5个板块，每板块30+道题，涵盖选择、填空、简答、算法设计四种题型。题目编排：先列全部题目，再统一给出答案解析，便于自测。

板块一 第1-2章（绪论+线性表）

一、选择题

1. 下列数据结构中，()属于非线性结构。

- A. 栈
- B. 队列
- C. 二叉树
- D. 串

2. 算法的时间复杂度取决于()。

- A. 问题的规模
- B. 待处理数据的初态
- C. A和B
- D. 计算机的配置

3. 以下算法的时间复杂度为()。

for(i=1; i<n; i++) for(j=1; j<i; j++) x++;

- A. $O(n)$
- B. $O(n \log n)$
- C. $O(n^2)$
- D. $O(n^3)$

4. 在长度为n的顺序表中查找一个元素，平均比较次数为()。

- A. n
- B. $(n+1)/2$
- C. $n/2$
- D. $(n-1)/2$

5. 在带头结点的单链表中，删除指针p所指结点的后继结点的操作是()。

- A. $p \rightarrow next = p \rightarrow next \rightarrow next$
- B. $p = p \rightarrow next \rightarrow next$
- C. $p \rightarrow next = p \rightarrow next$
- D. $p = p \rightarrow next$

6. 在长度为n的顺序表中插入一个元素，平均需要移动()个元素。

- A. $(n-1)/2$

- B. $n/2$
- C. $(n+1)/2$
- D. n

7. 单链表中，增加一个头结点的目的是为了()。

- A. 使单链表至少有一个结点
- B. 方便运算的实现
- C. 标识表结点中首结点的位置
- D. 说明单链表是线性的

8. 若线性表最常用的操作是存取任一指定序号的元素，最不常用的是插入删除，则采用()最节省时间。

- A. 顺序表
- B. 双向链表
- C. 带头结点的双循环链表
- D. 单链表

9. 对于双向链表，在p所指结点之后插入新结点s的正确操作是()。

- A. $p \rightarrow next = s$; $s \rightarrow prior = p$; $s \rightarrow next = p \rightarrow next$; $p \rightarrow next \rightarrow prior = s$
- B. $p \rightarrow next \rightarrow prior = s$; $p \rightarrow next = s$; $s \rightarrow prior = p$; $s \rightarrow next = p \rightarrow next$
- C. $s \rightarrow prior = p$; $s \rightarrow next = p \rightarrow next$; $p \rightarrow next \rightarrow prior = s$; $p \rightarrow next = s$
- D. $s \rightarrow next = p \rightarrow next$; $p \rightarrow next \rightarrow prior = s$; $s \rightarrow prior = p$; $p \rightarrow next = s$

10. 下面关于线性表的叙述中，正确的是()。

- A. 线性表的逻辑顺序与物理顺序总是一致的
- B. 线性表的顺序存储结构优于链式存储结构
- C. 线性表若采用链式存储则不必占用一片连续的存储空间
- D. 每种存储结构都各有其优缺点，应根据具体情况选择

11. 带头结点的单链表L为空的条件是()。

- A. $L == \text{NULL}$
- B. $L \rightarrow next == \text{NULL}$
- C. $L \rightarrow next == L$
- D. $L != \text{NULL}$

12. 在循环单链表中，若rear为尾指针，则判断表空的条件是()。

- A. $rear == \text{NULL}$
- B. $rear \rightarrow next == \text{NULL}$
- C. $rear \rightarrow next == rear$
- D. $rear \rightarrow next \rightarrow next == rear$

13. 在单循环链表中，若指针p所指结点是尾结点，则 $p \rightarrow next$ 的值是()。

- A. NULL

- B. p
- C. 头结点
- D. 不确定

14. 链表不具备的特点是()。

- A. 可随机访问任一结点
- B. 插入删除不需要移动元素
- C. 不必事先估计存储空间
- D. 所需空间与线性表长度成正比

15. 下列关于线性表的说法正确的是()。

- A. 线性表的元素个数不能为0
- B. 线性表中每个元素都有直接前驱和后继
- C. 线性表的逻辑结构与存储结构都唯一
- D. 线性表是有限序列

二、填空题

1. 数据结构按逻辑结构可分为线性结构和_____。

2. 算法的5个特性是：有穷性、确定性、可行性、输入和_____。

3. 在长度为n的顺序表中删除第i个元素，需要移动_____个元素。

4. 在带头结点的单链表中，要在第一个数据结点前插入新结点s，已知p指向头结点，则核心代码为_____。

5. 双向链表中，删除p所指结点的核心代码是_____。

6. 在循环队列中，用front和rear分别表示队头和队尾指针，MAXSIZE为最大容量，则队满条件是_____。

7. 单链表中设置头结点的作用是_____。

8. 顺序表中第i个元素的存储地址 $LOC(a_i)$ 与首元素 $LOC(a_1)$ 的关系是_____（设元素占c个存储单元）。

9. 在长度为n的顺序表中插入一个元素，平均移动次数为_____。

10. 数据的基本单位是_____。

三、简答题

1. 简述线性表的两种基本存储结构及其特点。

2. 简述头结点和头指针的区别。

3. 比较顺序表和链表在插入、删除操作上的时间复杂度。

4. 写出单链表逆置算法的思路。

四、算法设计题

1. 设计算法：将带头结点的单链表L逆置（要求原地工作，不申请新结点）。

```
void ReverseList(LinkList L) {
    if (!L->next || !L->next->next) return;
    LNode *p = L->next, *q;
    L->next = NULL;
    while (p) {
        q = p->next; // 暂存下一结点
        p->next = L->next; // 头插
        L->next = p;
        p = q;
    }
}
```

解析：思路：用头插法重建链表，原顺序被反转。需3个指针：p遍历原链表、q暂存p的后继、L->next始终指向新链表首结点。时间O(n)，空间O(1)。

2. 设计算法：在带头结点的单链表中，删除所有值为x的结点。

```
void DeleteX(LinkList L, ElemType x) {
    LNode *p = L, *q;
    while (p->next) {
        if (p->next->data == x) {
            q = p->next;
            p->next = q->next;
            free(q);
        } else {
            p = p->next;
        }
    }
}
```

解析：思路：用前驱指针p遍历，每次检查p->next；若值为x则删除并释放，p不动（因为新p->next是下一个待检查）；否则p前进。注意：删除后不要立即前进，因为新的p->next还未检查。时间O(n)。

3. 设计算法：将两个递增有序的单链表合并为一个递增有序的单链表（不申请新结点）。

```

LinkedList MergeList(LinkedList La, LinkedList Lb) {
    LNode *pa = La->next, *pb = Lb->next, *pc = La;
    La->next = NULL;
    while (pa && pb) {
        if (pa->data <= pb->data) {
            pc->next = pa; pc = pa; pa = pa->next;
        } else {
            pc->next = pb; pc = pb; pb = pb->next;
        }
    }
    pc->next = pa ? pa : pb; // 接上剩余
    free(Lb);
    return La;
}

```

解析：思路：双指针法。用La的头结点作为合并后链表的头结点，pc指针指向合并链表的尾结点。比较pa和pb数据，较小者链接到pc后。最后剩余部分直接接上。时间 $O(m+n)$ ，空间 $O(1)$ 。

【板块一参考答案与解析】

选择题答案

1. 答案：C。解析：栈、队列、串均为线性结构；二叉树为树形结构。
2. 答案：C。解析：时间复杂度既与问题规模有关，也受数据初态影响（如快排最坏情况）。
3. 答案：C。解析：内层执行 $\sum_{i=1}^{n-1} (i-1) = (n-1)(n-2)/2$ ，最高阶 $O(n^2)$ 。
4. 答案：B。解析：顺序查找 $ASL = (n+1)/2$ 。
5. 答案：A。解析：删除后继：p->next指向其后继的后继。还需free原后继。
6. 答案：B。解析：插入平均移动 $n/2$ （分母 $n+1$ ）。
7. 答案：B。解析：头结点简化首结点插入/删除的边界处理，避免特殊判断。
8. 答案：A。解析：顺序表随机存取 $O(1)$ ，链表需 $O(n)$ 。
9. 答案：C。解析：正确顺序：先处理s的两个指针，再修改p->next的前驱和p的next。
10. 答案：D。解析：A错（链表物理顺序可任意）；BC太绝对；D正确。
11. 答案：B。解析：头结点存在($L!=NULL$)，但其next为空表示无数据结点。
12. 答案：C。解析：循环链表尾指针指回头结点 $rear->next==rear$ (头)时为空。
13. 答案：C。解析：循环链表尾结点的next指回头结点。
14. 答案：A。解析：链表是顺序存取，不能随机访问。

15. 答案：D。 解析：A错(空表 $n=0$)；B错(首尾元素)；C错(逻辑唯一但存储多样)；D正确。

填空题答案

1. 非线性结构
2. 输出
3. $n-i$
4. $s \rightarrow next = p \rightarrow next; p \rightarrow next = s;$
5. $p \rightarrow prior \rightarrow next = p \rightarrow next; p \rightarrow next \rightarrow prior = p \rightarrow prior;$
6. $(rear+1) \% MAXSIZE == front$
7. 简化首结点操作的边界处理
8. $LOC(a_i) = LOC(a_1) + (i-1) * c$
9. $n/2$
10. 数据元素

简答题参考答案

1. 顺序存储：地址连续，随机存取 $O(1)$ ，插入删除需移动元素 $O(n)$ ，存储密度高；链式存储：地址不连续，顺序存取 $O(n)$ ，插入删除 $O(1)$ （已知位置），需额外指针空间。
2. 头指针：指向链表第一个结点的指针，链表必存在；头结点：在第一个数据结点前附设的结点，数据域可不放信息或存表长。带头结点时，头指针指向头结点；空表判断简化为头结点的next为NULL。
3. 顺序表：插入删除都需移动元素， $O(n)$ ；链表：已知插入/删除位置的前驱时，仅需修改指针， $O(1)$ 。但查找位置都需 $O(n)$ （顺序表可用折半查找 $O(\log n)$ ）。
4. 采用头插法重建：设三个指针p(当前结点)、q(暂存下一结点)；遍历原链表，对每个结点用头插法插入到头结点之后。原顺序被反转。时间 $O(n)$ ，空间 $O(1)$ 。

板块二 第3-4章（栈和队列+串）

一、选择题

1. 若进栈序列为1,2,3,4,5, 则下列不可能的出栈序列是()。
 - A. 3,2,1,4,5
 - B. 5,4,3,2,1
 - C. 1,2,3,4,5
 - D. 1,3,2,4,5

1. 若进栈序列为1,2,3,4,5, 则不可能的出栈序列是()。
 - A. 3,2,1,4,5
 - B. 5,4,1,2,3
 - C. 1,2,3,4,5
 - D. 1,3,2,4,5

2. 设计一个判别表达式中左右括号是否配对的算法, 采用()数据结构最佳。
 - A. 顺序表
 - B. 栈
 - C. 队列
 - D. 链表

3. 一个栈的入栈序列是a,b,c,d,e, 则栈的不可能输出序列是()。
 - A. edcba
 - B. decba
 - C. dceab
 - D. abcde

4. 循环队列的队满条件为()。
 - A. $front == rear$
 - B. $(rear+1) \% MAXSIZE == front$
 - C. $rear == MAXSIZE$
 - D. $rear == MAXSIZE-1$

5. 循环队列存储在数组A[0..M-1]中, front指向队头元素前一位置, rear指向队尾元素, 则队空条件是()。
 - A. $front == rear$
 - B. $(rear+1) \% M == front$
 - C. $rear == (front+1) \% M$
 - D. $front == (rear+1) \% M$

6. 若用数组A[0..5]来实现循环队列, 队头指针front=2, 队尾指针rear=5, 则队列中元素个数是()。
 - A. 2

- B. 3
- C. 4
- D. 5

7. 栈和队列的共同点是()。

- A. 都是先进先出
- B. 都是后进先出
- C. 只允许在端点处插入和删除
- D. 没有共同点

8. 在解决计算机主机与打印机之间速度不匹配问题时，通常设置一个打印数据缓冲区，该缓冲区采用的是()结构。

- A. 栈
- B. 队列
- C. 树
- D. 图

9. 两个栈共享一片连续内存空间时，为提高内存利用率，通常使两个栈的栈底分别设在空间的()。

- A. 中间和两端
- B. 同一端
- C. 两端
- D. 任意位置

10. 串的长度是指()。

- A. 串中所含不同字母的个数
- B. 串中所含字符的个数
- C. 串中所含不同字符的个数
- D. 串中非空格字符的个数

11. 空串与空格串是相同的，这种说法()。

- A. 正确
- B. 错误

12. 串"abcabaa"的next数组(严蔚敏版，next[1]=0)是()。

- A. 0112231
- B. 0111232
- C. 0123456
- D. 0112342

13. 串"abaabcac"的next数组(严蔚敏版)是()。

- A. 01122312
- B. 01121232
- C. 01232110

D. 01112321

14. KMP算法的关键是()。

- A. 计算next数组
- B. 主串指针回溯
- C. 减少比较次数
- D. 增加存储空间

15. 模式串"aaab"的next数组(严蔚敏版)是()。

- A. 0123
- B. 0111
- C. 0120
- D. 0112

二、填空题

1. 栈的特点是____，队列的特点是____。
2. 4个元素依次进栈，可能的出栈序列有____种。
3. 循环队列中，判空条件是____，判满条件是____。
4. 队列的队尾指针指向____。
5. 串"abcde"的长度是____。
6. 两个串相等的充要条件是____。
7. 模式串"ababaa"的next数组(严蔚敏版)是____。
8. KMP算法的时间复杂度是____。
9. 在栈S中，Pop操作前应判断栈是否为____。
10. 中缀表达式"3+4*2"转换为后缀表达式是____。

三、简答题

1. 简述栈和队列的异同。
2. 为什么循环队列要"牺牲"一个存储空间？
3. 简述KMP算法相比BF算法的改进。
4. 写出next数组的计算规则（严蔚敏版）。

四、算法设计题

1. 设计算法：用一个数组实现两个栈，要求充分利用数组空间。

```
#define MAXSIZE 100
typedef struct {
    ElemType data[MAXSIZE];
    int top1; // 栈1栈顶, 初始-1
    int top2; // 栈2栈顶, 初始MAXSIZE
} DStack;

Status Push(DStack *S, ElemType e, int stackNum) {
    if (S->top1 + 1 == S->top2) return ERROR; // 栈满
    if (stackNum == 1) S->data[++S->top1] = e;
    else S->data[--S->top2] = e;
    return OK;
}

Status Pop(DStack *S, ElemType *e, int stackNum) {
    if (stackNum == 1) {
        if (S->top1 == -1) return ERROR;
        *e = S->data[S->top1--];
    } else {
        if (S->top2 == MAXSIZE) return ERROR;
        *e = S->data[S->top2++];
    }
    return OK;
}
```

解析：思路：栈1从数组左端开始向右增长（top1初始-1），栈2从右端开始向左增长（top2初始MAXSIZE）。栈满条件：top1+1==top2。这样两栈共享空间，提高利用率。

2. 设计算法：用两个栈实现一个队列，支持入队和出队操作。

```
typedef struct {
    Stack S1; // 入队栈
    Stack S2; // 出队栈
} QueueByStack;

void Enqueue(QueueByStack *Q, ElemType e) {
    Push(&Q->S1, e);
}

Status Dequeue(QueueByStack *Q, ElemType *e) {
    if (StackEmpty(Q->S2)) {
        if (StackEmpty(Q->S1)) return ERROR; // 队列空
        while (!StackEmpty(Q->S1)) {
            Pop(&Q->S1, &x);
            Push(&Q->S2, x);
        }
    }
    return Pop(&Q->S2, e);
}
```

解析：思路：S1负责入队，S2负责出队。出队时若S2非空直接Pop；若S2空则把S1中所有元素依次Pop并Push到S2（顺序反转），再从S2 Pop。这样入队O(1)，出队均摊O(1)。

3. 设计算法：实现BF模式匹配。

```
int Index_BF(SString S, SString T, int pos) {
    int i = pos, j = 1;
    while (i <= S.length && j <= T.length) {
        if (S[i] == T[j]) {
            i++; j++;
        } else {
            i = i - j + 2; // 主串回溯到本次起点+1
            j = 1;
        }
    }
    if (j > T.length) return i - T.length;
    return 0;
}
```

解析：思路：双指针i(主串)、j(模式串)。逐字符比较，匹配则同进；失配则主串回到本次匹配起点+1，模式串重置。最坏 $O(n \times m)$ 。

【板块二参考答案与解析】

选择题答案

1. B 5最先出栈意味着1-5全部入栈，5出栈后4必先于3、3先于2、2先于1，应为5,4,3,2,1。其实B是可能的。让我们重新看：5出栈→4出栈→3出栈→2出栈→1出栈，B是可能的。改选D：1出后2/3/4/5任一可出。D也合法。本题若选最不可能，则B对应1,2,3,4,5依次入后5先出，需要全部入栈，可能。换一题。
2. B B中5先出需全入，4再出，此时栈中1,2,3（底到顶），下次应出3，不可能出1。故B不可能。
3. C 括号匹配需"后进先出"，栈最合适。
4. B C中d先出需abcd入栈，d出后栈内abc；下一步出c合法；但下一步要出a，而a上方有b，不可能直接出a。
5. A 牺牲一个空间判满： $(rear+1) \% MAXSIZE == front$ 。
6. B $front == rear$ 表示队空。
7. C $(rear - front + M) \% M = (5 - 2 + 6) \% 6 = 3$ 。
8. B 栈和队列都是操作受限的线性表，仅允许在端点操作。
9. C 打印机需要按提交顺序打印，先进先出，用队列。
10. B 两栈底设在空间两端，栈顶向中间延伸，可减少溢出。
11. B 串长度=所含字符个数（含空格）。
12. B 空串长度为0；空格串长度为空格数。
13. A 逐位计算： $a \rightarrow 0, ab \rightarrow 1, abc \rightarrow 1, abca \rightarrow 2, abcab \rightarrow 2, abcaba \rightarrow 3, abcabaa \rightarrow 2$ 。
14. A $a \rightarrow 0, ab \rightarrow 1, aba \rightarrow 1, abaa \rightarrow 2, abaab \rightarrow 2, abaabc \rightarrow 3, abaabca \rightarrow 1, abaabcac \rightarrow 2$ 。
15. A KMP核心是next数组预处理，使主串指针不回溯。

填空题答案

1. 后进先出(LIFO)；先进先出(FIFO)
2. 14（卡特兰数 $C_4 = 14$ ）

3. $\text{front} == \text{rear}; (\text{rear} + 1) \% \text{MAXSIZE} == \text{front}$
4. 队尾元素的下一位置（或最后一个元素，取决于定义）
5. 5
6. 长度相等且对应位置字符相同
7. 0,1,1,2,3,1
8. $O(n+m)$ （ n 主串长， m 模式串长）
9. 空
10. 3 4 2 * +

简答题参考答案

1. 相同点：都是操作受限的线性表，都只能在线性表端点进行操作。不同点：栈只允许在同一端（栈顶）插入删除，LIFO；队列允许在一端（队尾）插入、另一端（队头）删除，FIFO。
2. 若不牺牲， $\text{front} == \text{rear}$ 既可能是空也可能是满，无法区分。牺牲一个空间后，判满条件变为 $(\text{rear} + 1) \% M == \text{front}$ ，与判空 $\text{front} == \text{rear}$ 区分开。代价是实际容量为 $M-1$ 。
3. BF算法主串指针在失配时需回溯，最坏 $O(n \times m)$ 。KMP通过预处理模式串得到next数组，失配时主串指针不回溯，仅模式串指针根据next跳转，最坏 $O(n+m)$ 。next数组记录了模式串每个位置的最长相等前后缀信息。
4. ① $\text{next}[1] = 0$ ；② $\text{next}[2] = 1$ （单字符无前后缀）；③ 对 $j \geq 3$ ，看模式串 $T[1..j-1]$ 的最长相等前后缀长度 k ， $\text{next}[j] = k + 1$ ；④ 若无相等前后缀， $\text{next}[j] = 1$ 。

板块三 第5-6章（数组广义表+树和二叉树）

一、选择题

1. 广义表(a, (b, c, d), e)的长度是()。

- A. 2
- B. 3
- C. 4
- D. 5

2. 广义表(a, (b, c, d))的表头是()。

- A. a
- B. (a)
- C. (b,c,d)
- D. b

3. 广义表(a, (b, c, d))的表尾是()。

- A. (b,c,d)
- B. ((b,c,d))
- C. b,c,d
- D. (a)

4. 广义表((a,b),(c,d),(e,f))的深度是()。

- A. 1
- B. 2
- C. 3
- D. 4

5. 已知二维数组A[0..4][0..5]按行主序存储，每个元素占4个字节，A[0][0]地址为1000，则A[3][4]的地址是()。

- A. 1088
- B. 1092
- C. 1096
- D. 1100

5. 已知二维数组A[0..4][0..5]按行主序存储，每个元素占4字节，A[0][0]地址为1000，A[3][4]的地址是()。

- A. 1088
- B. 1092
- C. 1096
- D. 1100

6. n阶对称矩阵按行主序压缩存储到一维数组B（下标从1开始），元素a[i][j]（ $i \geq j$ ）在B中的下标是()（i,j从1开始）。

- A. $i*(i-1)/2+j$
 - B. $i*(i-1)/2+j-1$
 - C. $i*(i+1)/2+j$
 - D. $j*(j-1)/2+i$
7. 深度为 k ($k \geq 1$) 的满二叉树共有()个结点。
- A. 2^k
 - B. 2^{k-1}
 - C. $2^{(k-1)}$
 - D. $2^{(k+1)}$
8. 一棵完全二叉树有1001个结点, 其叶子结点个数是()。
- A. 500
 - B. 501
 - C. 502
 - D. 503
9. 在二叉树的第 k 层上至多有()个结点 ($k \geq 1$) 。
- A. 2^k
 - B. $2^{(k-1)}$
 - C. $2^{(k+1)}$
 - D. 2^k-1
10. n 个结点的二叉链表中, 空指针数为()。
- A. $n-1$
 - B. n
 - C. $n+1$
 - D. $n+2$
11. 一棵二叉树中度为2的结点数为5, 度为1的结点数为3, 则叶子结点数为()。
- A. 5
 - B. 6
 - C. 7
 - D. 8
12. 前序遍历和中序遍历序列相同的二叉树为()。
- A. 根结点无左子树
 - B. 根结点无右子树
 - C. 只有根结点的二叉树
 - D. 空树或所有的结点都没有左子树
13. 一棵二叉树前序为ABDECFG, 中序为DBEAF CG, 则后序为()。
- A. DEBFGCA

- B. DBFGECA
- C. DEBFGCA
- D. DEFGBCA

14. 哈夫曼树没有度为()的结点。

- A. 0
- B. 1
- C. 2
- D. 3

15. n 个叶子结点的哈夫曼树共有()个结点。

- A. $2n$
- B. $2n-1$
- C. $2n+1$
- D. $n+1$

二、填空题

1. 广义表(a, (b, c, (d)))的深度是_____。
2. 广义表 $L=((a,b),c,(d,e,f))$, $\text{head}(\text{tail}(L))=$ _____。
3. 对称矩阵压缩存储, $n \times n$ 矩阵需要_____个存储单元。
4. 深度为 k 的二叉树最多有_____个结点。
5. 具有 n 个结点的完全二叉树的深度为_____。
6. n 个结点的二叉链表共有_____个空指针域。
7. 二叉树的第 i 层最多有_____个结点 ($i \geq 1$) 。
8. 已知二叉树前序ABDCE, 中序BDAEC, 则后序是_____。
9. 哈夫曼树是带权路径长度_____的二叉树。
10. 在线索二叉树中, $\text{ltag}=1$ 表示 lchild 是_____。

三、简答题

1. 简述二叉树的5个性质。
2. 什么是线索二叉树? 为什么要线索化?
3. 简述哈夫曼树的构造过程。

4. 写出由先序和中序序列构造二叉树的算法思路。

四、算法设计题

1. 设计算法：求二叉树中叶子结点的个数。

```
int CountLeaf(BiTree T) {
    if (T == NULL) return 0;
    if (T->lchild == NULL && T->rchild == NULL) return 1;
    return CountLeaf(T->lchild) + CountLeaf(T->rchild);
}
```

解析：思路：递归。空树0个叶子；左右孩子都空则是叶子，返回1；否则返回左右子树叶子数之和。时间 $O(n)$ 。

2. 设计算法：求二叉树的深度。

```
int BiTreeDepth(BiTree T) {
    if (T == NULL) return 0;
    int ldep = BiTreeDepth(T->lchild);
    int rdep = BiTreeDepth(T->rchild);
    return (ldep > rdep ? ldep : rdep) + 1;
}
```

解析：思路：递归。空树深度0；否则取左右子树深度的较大值+1。时间 $O(n)$ 。

3. 设计算法：实现二叉树的中序非递归遍历（用栈）。

```
void InOrder_NR(BiTree T) {
    Stack S; InitStack(S);
    BiTree p = T;
    while (p || !StackEmpty(S)) {
        if (p) {
            Push(S, p);
            p = p->lchild; // 一路向左入栈
        } else {
            Pop(S, p);
            visit(p); // 访问栈顶
            p = p->rchild; // 转向右子树
        }
    }
}
```

解析：思路：用栈模拟递归。当前结点非空则入栈并转向左孩子；为空则弹栈访问，转向右孩子。直到栈空且 p 为NULL。时间 $O(n)$ ，空间 $O(h)$ ， h 为树高。

4. 设计算法：判断两棵二叉树是否相同（结构和值都相同）。

```
bool IsSame(BiTree T1, BiTree T2) {
    if (T1 == NULL && T2 == NULL) return true;
    if (T1 == NULL || T2 == NULL) return false;
    if (T1->data != T2->data) return false;
    return IsSame(T1->lchild, T2->lchild) && IsSame(T1->rchild, T2->rchild);
}
```

解析：思路：递归。两棵都空则相同；一空一非空则不同；根值不等则不同；递归判断左右子树是否相同。时间 $O(\min(n,m))$ 。

【板块三参考答案与解析】

选择题答案

1. B 最外层元素：a、(b,c,d)、e，共3个。
2. A 表头=第一个元素=a。
3. B 表尾=剩余元素组成的新表，必须加外层括号。
4. B 括号嵌套两层，深度=2。
5. A 地址= $1000+((3-0)*6+(4-0))*4=1000+(18+4)*4=1000+88=1088$ 。等等，重算：
 $1000+((3)*6+(4))*4=1000+22*4=1088$ 。选A。
6. A 地址= $1000+(3*6+4)*4=1000+22*4=1088$ 。
7. B $i \geq j$ 时存储在下三角，前面有 $i-1$ 行共 $(i-1)*i/2$ 个元素，第 i 行第 j 列前有 $j-1$ 个元素，下标
 $=(i-1)*i/2+(j-1)+1=i*(i-1)/2+j$ （B下标从1）。
8. B 满二叉树结点数= 2^k-1 （ k 为深度）。
9. B $n=1001$, $n_0=(n+1)/2=501$ （ n 为奇数时叶子数 $=(n+1)/2$ ）。
10. C 第 k 层最多 2^{k-1} 个结点。
11. B $2n$ 个指针域， $n-1$ 个非空（每非根结点被指一次），空指针 $=2n-(n-1)=n+1$ 。
12. D $n_0=n_2+1=5+1=6$ 。
13. A 前序DLR与中序LDR相同，说明没有L，即所有结点都无左子树。空树也满足。
14. B 前序首A为根；中序DBEAF CG中A左DBE为左子树，FCG为右子树；递归分析：左子树前序
BDE→根B，中序DBE→左D右E；右子树前序CFG→根C，中序FCG→左F右G。后序：DEBFGCA。
15. B 哈夫曼树每次合并两棵子树，故只有度为0（叶子）和2的结点。

填空题答案

1. 3
2. c
3. $n(n+1)/2$
4. 2^k-1
5. $\lfloor \log_2 n \rfloor + 1$
6. $n+1$
7. 2^{i-1}
8. DBECA
9. 最小
10. 线索（指向前驱）

简答题参考答案

1. ① 第 i 层最多 2^{i-1} 个结点；② 深度 k 最多 2^k-1 个结点；③ $n_0=n_2+1$ ；④ n 个结点完全二叉树深度
 $\lfloor \log_2 n \rfloor + 1$ ；⑤ 完全二叉树结点 i 的双亲 $\lfloor i/2 \rfloor$ ，左孩子 $2i$ ，右孩子 $2i+1$ 。

2. 线索二叉树：利用二叉链表中的 $n+1$ 个空指针域，存放结点在某种遍历序列中的前驱或后继指针（线索）。线索化的目的：①充分利用空指针域；②使遍历不用栈/递归，提高效率；③便于查找前驱后继。

3. ①将 n 个权值作为 n 棵只有根结点的树构成森林 F ；②在 F 中选两棵根权值最小的树合并为新树，新树根权=两子树根权之和；③从 F 删除这两棵树，加入新树；④重复②③直到 F 中只剩一棵树。

4. 先序首元素为根，在中序中找到根的位置 i ， i 左侧为左子树中序（共 $i-1$ 个元素），右侧为右子树中序。在先序中，根之后的 $i-1$ 个元素为左子树先序，剩余为右子树先序。递归处理左右子树即可。

板块四 第7章（图）

一、选择题

1. n 个顶点的无向完全图有()条边。

- A. $n(n-1)$
- B. $n(n-1)/2$
- C. n^2
- D. $n^2/2$

2. n 个顶点的有向完全图有()条弧。

- A. $n(n-1)$
- B. $n(n-1)/2$
- C. n^2
- D. n

3. 无向图的邻接矩阵是()。

- A. 对称矩阵
- B. 上三角矩阵
- C. 下三角矩阵
- D. 对角矩阵

4. 有向图的邻接矩阵中，第 i 行的非零元素个数为第 i 个顶点的()。

- A. 度
- B. 入度
- C. 出度
- D. 度数减1

5. 对于一个有 n 个顶点 e 条边的无向图，其邻接表有()个边结点。

- A. n
- B. e
- C. $2e$
- D. $n+e$

6. 图的广度优先搜索遍历类似于树的()遍历。

- A. 先根
- B. 中根
- C. 后根
- D. 层序

7. 图的深度优先搜索遍历类似于树的()遍历。

- A. 先根
- B. 中根

- C. 后根
 - D. 层序
8. 无向图的邻接表中, 顶点v的边链表中结点的个数等于()。
- A. 顶点v的度
 - B. 顶点v的入度
 - C. 顶点v的出度
 - D. 顶点v的度的一半
9. 求单源最短路径的Dijkstra算法, 其时间复杂度为()。
- A. $O(n)$
 - B. $O(n^2)$
 - C. $O(n^3)$
 - D. $O(n \log n)$
10. 下列关于Dijkstra算法的说法错误的是()。
- A. 不能用于负权图
 - B. 既能用于有向图也能用于无向图
 - C. 求单源最短路径
 - D. 时间复杂度 $O(n^3)$
11. 对于n个顶点e条边的连通图, 其最小生成树有()条边。
- A. n
 - B. n-1
 - C. n+1
 - D. e
12. Prim算法构造最小生成树适合于()。
- A. 稠密图
 - B. 稀疏图
 - C. 完全图
 - D. 任意图
13. Kruskal算法构造最小生成树的时间复杂度为()。
- A. $O(n^2)$
 - B. $O(e \log e)$
 - C. $O(n^3)$
 - D. $O(n \log n)$
14. 在有向图的邻接表存储中, 顶点v的边链表中结点的个数等于()。
- A. 顶点v的度
 - B. 顶点v的入度
 - C. 顶点v的出度

D. 顶点v的入度加出度

15. 一个有向图的拓扑排序序列()。

- A. 一定唯一
- B. 一定不唯一
- C. 可能有多个
- D. 一定不存在

16. 判断一个有向图是否有回路, 可以用()。

- A. 求最短路径
- B. 求最小生成树
- C. 拓扑排序
- D. DFS

17. AOV网是一种()。

- A. 有向无环图
- B. 无向图
- C. 完全图
- D. 强连通图

18. 在图的最短路径算法中, ()算法可求解任意两顶点间的最短路径。

- A. Dijkstra
- B. Floyd
- C. BFS
- D. DFS

二、填空题

1. 图的存储结构主要有邻接矩阵、邻接表、____和邻接多重表。
2. n 个顶点的无向图, 邻接矩阵中非零元素个数是边数的____倍。
3. 深度优先搜索遍历图类似于树的____遍历。
4. 广度优先搜索遍历图用____辅助实现。
5. DFS遍历图用____辅助实现。
6. 在邻接表存储的有向图有 n 个顶点 e 条弧, 则DFS时间复杂度为____。
7. 在邻接矩阵存储的有向图有 n 个顶点 e 条弧, 则DFS时间复杂度为____。
8. Dijkstra算法不能用于____权的图。
9. n 个顶点的连通图至少有____条边。

10. AOV网的拓扑排序目的是判断图中是否有_____。

三、简答题

1. 简述邻接矩阵和邻接表的特点及适用场景。
2. 简述DFS和BFS的算法思路及时间复杂度。
3. 简述Prim和Kruskal算法的异同。
4. 什么是拓扑排序？如何判断有向图是否有环？

四、算法设计题

1. 设计算法：用邻接表存储的无向图，求顶点v的度。

```
int VertexDegree(ALGraph G, int v) {
    int degree = 0;
    ArcNode *p = G.vertices[v].firstarc;
    while (p != NULL) {
        degree++;
        p = p->nextarc;
    }
    return degree;
}
```

解析：思路：邻接表中顶点v的边链表结点数=v的度。遍历链表计数即可。时间O(度数)。

2. 设计算法：基于邻接表实现DFS遍历。

```
bool visited[MAX];
void DFS(ALGraph G, int v) {
    visit(v); // 访问顶点v
    visited[v] = true;
    ArcNode *p = G.vertices[v].firstarc;
    while (p != NULL) {
        int w = p->adjvex;
        if (!visited[w])
            DFS(G, w);
        p = p->nextarc;
    }
}

void DFSTraverse(ALGraph G) {
    for (int i = 0; i < G.vexnum; i++) visited[i] = false;
    for (int i = 0; i < G.vexnum; i++)
        if (!visited[i]) DFS(G, i);
}
```

解析：思路：递归DFS。从顶点v出发，访问之并标记，然后对其每个未访问的邻接点递归DFS。DFSTraverse处理非连通图（多次调用DFS直到所有顶点访问）。时间O(n+e)。

3. 设计算法：基于邻接表实现BFS遍历。

```

void BFS(ALGraph G, int v) {
    Queue Q; InitQueue(Q);
    visit(v); visited[v] = true;
    EnQueue(Q, v);
    while (!QueueEmpty(Q)) {
        int u; DeQueue(Q, u);
        ArcNode *p = G.vertices[u].firstarc;
        while (p != NULL) {
            int w = p->adjvex;
            if (!visited[w]) {
                visit(w); visited[w] = true;
                EnQueue(Q, w);
            }
            p = p->nextarc;
        }
    }
}

```

解析：思路：队列辅助。从v出发访问之并入队；队头出队，遍历其所有邻接点，未访问者访问并入队；直到队空。逐层扩展。时间 $O(n+e)$ 。

4. 设计算法：判断无向图G是否连通（基于DFS）。

```

bool visited[MAX];
bool IsConnected(ALGraph G) {
    for (int i = 0; i < G.vexnum; i++) visited[i] = false;
    DFS(G, 0); // 从顶点0开始DFS
    for (int i = 0; i < G.vexnum; i++)
        if (!visited[i]) return false; // 有顶点未访问
    return true;
}

```

解析：思路：从任一顶点DFS，若能访问所有顶点则连通，否则不连通。时间 $O(n+e)$ 。

【板块四参考答案与解析】

选择题答案

1. B 无向完全图边数 $=n(n-1)/2$ 。
2. A 有向完全图弧数 $=n(n-1)$ 。
3. A 无向图边无方向， $A[i][j]=A[j][i]$ ，故对称。
4. C 邻接矩阵第i行表示从顶点i出发的弧，故为出度。
5. C 无向图每条边在两个顶点的链表中各出现一次，共 $2e$ 个边结点。
6. D BFS逐层扩展，类似树的层序遍历。
7. A DFS一路深入，类似树的先根遍历。
8. A 无向图不区分出入度，边链表结点数=度。
9. B Dijkstra时间复杂度 $O(n^2)$ 。
10. D Dijkstra时间复杂度 $O(n^2)$ ，不是 $O(n^3)$ 。Floyd才是 $O(n^3)$ 。
11. B 生成树是极小连通子图，含n个顶点n-1条边。
12. A Prim $O(n^2)$ ，适合稠密图；Kruskal适合稀疏图。

13. B Kruskal主要花费在排序边, $O(e \log e)$ 。
14. C 邻接表存储出边, 每个边结点对应一条出边。
15. C 拓扑排序序列可能不唯一 (多个入度为0的顶点可任选)。
16. C 拓扑排序若不能输出所有顶点, 则存在回路。
17. A AOV网是有向无环图 (DAG)。
18. B Floyd算法求多源最短路径, 时间 $O(n^3)$ 。

填空题答案

1. 十字链表
2. 2
3. 先根
4. 队列
5. 栈
6. $O(n+e)$
7. $O(n^2)$
8. 负
9. $n-1$
10. 回路(环)

简答题参考答案

1. 邻接矩阵: $n \times n$ 矩阵, $A[i][j]$ 表示边是否存在; 适合稠密图, 空间 $O(n^2)$; 查邻接点需扫描一行 $O(n)$ 。邻接表: 每个顶点一个链表存邻接点; 适合稀疏图, 空间 $O(n+e)$; 查邻接点直接遍历链表。
2. DFS: 从某顶点出发访问之, 然后从未访问的邻接点出发递归DFS, 用栈 (递归调用栈) 辅助。BFS: 从某顶点出发访问之, 然后依次访问其所有未访问邻接点, 再用这些邻接点依序扩展, 用队列辅助。邻接矩阵存储 $O(n^2)$, 邻接表存储 $O(n+e)$ 。
3. 相同点: 都构造最小生成树, 都基于贪心策略。不同点: Prim从顶点出发, 每次选横跨已选和未选的最小边, 逐步扩展; $O(n^2)$, 适合稠密图。Kruskal从边出发, 按权值排序后依次选不构成回路的最小边; $O(e \log e)$, 适合稀疏图, 需并查集判回路。
4. 拓扑排序: 对AOV网进行排序, 使每个活动的前驱都排在其前面。算法: 每次选入度为0的顶点输出并删除其出边, 重复。若能输出所有顶点则无环; 若剩余顶点入度均不为0则存在环。

板块五 第9-10章（查找+排序）

一、选择题

1. 在含有 n 个关键字的二叉排序树上查找一个关键字，最坏情况时间复杂度为()。
A. $O(n)$
B. $O(\log n)$
C. $O(n \log n)$
D. $O(n^2)$
2. 折半查找的时间复杂度为()。
A. $O(n)$
B. $O(\log n)$
C. $O(n \log n)$
D. $O(1)$
3. 折半查找要求线性表必须是()。
A. 顺序存储的有序表
B. 链式存储的有序表
C. 顺序存储的无序表
D. 链式存储的无序表
4. 在平衡二叉树（AVL）中，任一结点的平衡因子取值范围是()。
A. -1, 0, 1
B. 0, 1, 2
C. -2, -1, 0, 1, 2
D. 任意整数
5. AVL树在插入新结点后，若发生失衡，需要()调整。
A. 旋转
B. 重新构造
C. 删除多余结点
D. 改变结点颜色
6. 在AVL树中插入一个结点导致失衡，若失衡结点的平衡因子为2，且其左孩子平衡因子为1，则应进行()旋转。
A. LL
B. RR
C. LR
D. RL
7. 哈希查找中，解决冲突的常用方法有()。
A. 顺序查找法

- B. 折半查找法
 - C. 线性探测法
 - D. 分块查找法
8. 哈希表的查找效率主要取决于()。
- A. 哈希函数
 - B. 处理冲突的方法
 - C. 装填因子
 - D. 以上都是
9. 下列排序算法中, 不稳定的是()。
- A. 冒泡排序
 - B. 直接插入排序
 - C. 快速排序
 - D. 归并排序
10. 下列排序算法中, 时间复杂度为 $O(n \log n)$ 且空间复杂度为 $O(1)$ 的是()。
- A. 堆排序
 - B. 快速排序
 - C. 归并排序
 - D. 冒泡排序
11. 快速排序在最坏情况下的时间复杂度为()。
- A. $O(n \log n)$
 - B. $O(n^2)$
 - C. $O(n)$
 - D. $O(n^3)$
12. 在下列排序算法中, 关键字比较次数与记录的初始排列无关的是()。
- A. 直接插入排序
 - B. 冒泡排序
 - C. 简单选择排序
 - D. 快速排序
13. 堆排序中, 建堆的时间复杂度为()。
- A. $O(n)$
 - B. $O(n \log n)$
 - C. $O(n^2)$
 - D. $O(\log n)$
14. 一组记录的关键码为(46,79,56,38,40,84), 则建堆后的大顶堆是()。
- A. 84,79,56,46,40,38
 - B. 84,79,56,38,40,46

- C. 84,56,79,40,46,38
- D. 79,46,56,38,40,84

15. 归并排序的辅助空间复杂度为()。

- A. $O(1)$
- B. $O(\log n)$
- C. $O(n)$
- D. $O(n^2)$

16. 下列排序算法中, ()是交换排序。

- A. 直接插入排序
- B. 简单选择排序
- C. 冒泡排序
- D. 归并排序

17. 希尔排序属于()。

- A. 插入排序
- B. 交换排序
- C. 选择排序
- D. 归并排序

18. 一组记录关键字为(25,48,16,35,79,80,30,40), 对其进行初始堆(大顶堆)后, 堆顶元素为()。

- A. 25
- B. 48
- C. 80
- D. 79

二、填空题

1. 在长度为n的顺序表中, 顺序查找的平均查找长度ASL=_____。
2. 折半查找的平均查找长度ASL约为_____。
3. 在二叉排序树中查找一个关键字的过程, 类似于在二叉树上进行_____查找。
4. AVL树的四种失衡类型是LL、RR、_____和RL。
5. 哈希表解决冲突的常用方法有开放定址法和_____。
6. 快速排序在最坏情况下的时间复杂度是_____。
7. 堆排序在建堆时, 从第_____个元素开始向下调整。
8. 排序算法的稳定性是指_____。

9. 归并排序的空间复杂度是_____。

10. 直接插入排序在最坏情况下的时间复杂度是_____。

三、简答题

1. 什么是哈希冲突？如何减少冲突？
2. 简述AVL树四种旋转的适用场景。
3. 简述快速排序的基本思想和最坏情况。
4. 比较直接插入排序、冒泡排序、简单选择排序的性能。

四、算法设计题

1. 设计算法：实现折半查找（递归版）。

```
int Search_Bin_Rec(SSTable ST, KeyType key, int low, int high) {
    if (low > high) return 0; // 查找失败
    int mid = (low + high) / 2;
    if (ST.elem[mid].key == key) return mid;
    else if (key < ST.elem[mid].key)
        return Search_Bin_Rec(ST, key, low, mid - 1);
    else
        return Search_Bin_Rec(ST, key, mid + 1, high);
}
```

解析：思路：每次比较中间元素，相等返回；小于则递归左半区；大于递归右半区；low>high返回0表示失败。时间 $O(\log n)$ ，空间 $O(\log n)$ 递归栈。

2. 设计算法：实现快速排序（含Partition函数）。

```
int Partition(int A[], int low, int high) {
    int pivot = A[low];
    while (low < high) {
        while (low < high && A[high] >= pivot) high--;
        A[low] = A[high];
        while (low < high && A[low] <= pivot) low++;
        A[high] = A[low];
    }
    A[low] = pivot;
    return low;
}

void QuickSort(int A[], int low, int high) {
    if (low < high) {
        int pivotpos = Partition(A, low, high);
        QuickSort(A, low, pivotpos - 1);
        QuickSort(A, pivotpos + 1, high);
    }
}
```

解析：思路：Partition选A[low]为pivot，用双指针从两端扫描，把比pivot小的移到左侧，大的移到右侧，最后pivot就位。QuickSort递归处理左右两部分。平均 $O(n \log n)$ ，最坏 $O(n^2)$ 。

3. 设计算法：实现堆排序的向下调整函数 sift，以及堆排序主函数。

```
void sift(int A[], int low, int high) {
    int i = low, j = 2 * i; // j为i的左孩子
    while (j <= high) {
        if (j < high && A[j] < A[j+1]) j++; // j指向较大孩子
        if (A[i] >= A[j]) break; // 已是堆
        int tmp = A[i]; A[i] = A[j]; A[j] = tmp; // 交换
        i = j; j = 2 * i;
    }
}

void HeapSort(int A[], int n) {
    int i, tmp;
    for (i = n/2; i >= 1; i--) sift(A, i, n); // 建堆
    for (i = n; i > 1; i--) {
        tmp = A[1]; A[1] = A[i]; A[i] = tmp; // 堆顶换到末尾
        sift(A, 1, i-1); // 调整剩余
    }
}
```

解析：思路：sift：比较结点i与其较大孩子，若i小于较大孩子则交换，i下移，直到满足堆性质。
HeapSort：先从n/2向前调用sift建堆；然后每次将堆顶(最大)换到末尾，对剩余元素调用sift调整。
时间 $O(n \log n)$ ，空间 $O(1)$ 。

4. 设计算法：实现直接插入排序。

```
void InsertSort(int A[], int n) {
    int i, j;
    for (i = 2; i <= n; i++) {
        if (A[i] < A[i-1]) {
            A[0] = A[i]; // A[0]作哨兵
            for (j = i-1; A[j] > A[0]; j--)
                A[j+1] = A[j]; // 后移
            A[j+1] = A[0]; // 插入
        }
    }
}
```

解析：思路：从第2个元素开始，每个元素与前面有序区从后向前比较。若小于前驱则暂存到A[0]，前面元素后移，直到找到位置插入。最坏 $O(n^2)$ ，最好 $O(n)$ （已有序）。稳定。

【板块五参考答案与解析】

选择题答案

1. A 最坏情况BST退化为链表，查找 $O(n)$ 。
2. B 折半查找每次缩小一半， $O(\log n)$ 。
3. A 折半查找需要随机存取（顺序存储）且有序。
4. A AVL定义：任意结点平衡因子绝对值 ≤ 1 。
5. A AVL通过旋转恢复平衡。

6. A $BF=2$ (左高) + 左孩子 $BF=1$ (左高) = LL型, 右旋一次。
7. C 线性探测法是开放定址法的一种, 用于解决冲突。
8. D 哈希查找效率受哈希函数、冲突方法、装填因子共同影响。
9. C 快排不稳定; 插、冒、归并稳定。
10. A 堆排时间 $O(n \log n)$, 空间 $O(1)$ 。快排空间 $O(\log n)$, 归并 $O(n)$, 冒泡 $O(n^2)$ 。
11. B 快排最坏 (已有序) 退化为 $O(n^2)$ 。
12. C 简单选择排序每趟固定比较 $n-i$ 次, 与初始顺序无关。
13. A 建堆从 $n/2$ 开始向下调整, 总时间 $O(n)$ 。
14. A 从 $i=3$ 开始调整: $i=3(38,84) \rightarrow$ 交换; $i=2(79,56)$ 不动; $i=1(46,79,84) \rightarrow 84$ 最大上浮, 46下沉到原84位置(右孩子位置), 46的孩子为空。结果: 84,79,56,38,40,46。
15. C 归并排序需要 $O(n)$ 辅助数组。
16. C 冒泡排序通过相邻元素交换实现。
17. A 希尔排序是分组插入排序。
18. C 大顶堆堆顶为最大值80。

填空题答案

1. $(n+1)/2$
2. $\log_2(n+1)-1$
3. 折半
4. LR
5. 链地址法
6. $O(n^2)$
7. $\lfloor n/2 \rfloor$
8. 相同关键字的记录排序前后相对位置不变
9. $O(n)$
10. $O(n^2)$

简答题参考答案

1. 哈希冲突: 不同关键字通过哈希函数计算得到相同的哈希地址。减少冲突方法: ①选择好的哈希函数 (如除留余数法, p 取不大于表长的最大质数); ②减小装填因子 α ($\alpha = \text{填入表中元素数} / \text{表长}$, 通常 $\alpha \leq 0.75$); ③采用合适的冲突解决方法 (链地址法优于开放定址法)。
2. LL型: 新结点插入在失衡结点左子树的左子树, 右旋一次; RR型: 插入在右子树的右子树, 左旋一次; LR型: 插入在左子树的右子树, 先对左子树左旋 (变LL), 再对失衡结点右旋; RL型: 插入在右子树的左子树, 先对右子树右旋 (变RR), 再对失衡结点左旋。
3. 基本思想: 选一个基准 $pivot$, 通过一趟划分将序列分为两部分 (左侧 $\leq pivot$, 右侧 $\geq pivot$), 基准就位; 然后递归对左右两部分排序。最坏情况: 序列已有序 (或逆序) 且每次选首元素为基准, 每次划分极度不平衡, 退化为 $O(n^2)$ 。可通过选中间元素或随机元素作 $pivot$ 缓解。
4. 三者最坏和平均时间均为 $O(n^2)$ 。直接插入: 最好 $O(n)$ (已有序), 稳定; 冒泡: 最好 $O(n)$ (加标志位), 稳定; 简单选择: 最好最坏均为 $O(n^2)$ (比较次数固定), 不稳定, 但交换次数最少 (最多 $n-1$ 次)。直接插入适合基本有序数据, 简单选择适合交换代价大的场景。

第四部分 考前1小时冲刺页 + 真题B卷全解附录

1. 考前1小时冲刺页（20个核心考点）

进考场前最后浏览，每个考点配公式/口诀：

#	考点	核心要点
1	卡特兰数	$C_n=(1/(n+1))C(2n,n)$; $C_1=1, C_2=2, C_3=5, C_4=14, C_5=42$ 。出栈序列数。
2	顺序表平均移动	插入 $n/2$; 删除 $(n-1)/2$ 。分母 $n+1$ vs n 。
3	KMP的next首位	严蔚敏版 $next[1]=0$; 通用版 -1 。本课程用前者。
4	广义表表尾	必须加外层括号 。 $(a,(b,c))$ 的 $tail=((b,c))$ ，不是 (b,c) 。
5	二叉链表空指针	n 个结点 $\rightarrow n+1$ 个空指针（推导： $2n-(n-1)$ ）。
6	满二叉树结点	深度 $k \rightarrow 2^k-1$ 个结点。
7	二叉树度数关系	$n_0 = n_2 + 1$ （叶子=度2+1）。
8	无向图邻接表	n 点 m 边 $\rightarrow 2m$ 个边结点。有向图为 m 。
9	完全图边数	无向 $n(n-1)/2$; 有向 $n(n-1)$ 。
10	循环队列	判空 $front==rear$; 判满 $(rear+1)\%M==front$ 。
11	DFS/BFS辅助	DFS用 栈 ，BFS用 队列 。邻接表 $O(n+e)$ ，矩阵 $O(n^2)$ 。
12	Dijkstra	不适用负权 ；时间 $O(n^2)$ ；单源最短路径。
13	MST算法	Prim $O(n^2)$ 稠密图；Kruskal $O(e \log e)$ 稀疏图。
14	AVL四种旋转	LL右旋；RR左旋；LR先左后右；RL先右后左。
15	BST性质	中序遍历得 递增序列 。
16	哈夫曼树	无度1结点；总结点 $2n-1$ ； $WPL=\sum \text{叶子权} \times \text{路径长}$ 。
17	哈希ASL	成功 $=\sum P_i C_i / n$ ；失败 $=\sum C_i / \text{表长}$ 。装填因子 $\alpha=n/m$ 。
18	排序稳定性	插冒归基稳 （直接插入、冒泡、归并、基数）； 希快选堆不稳 （希尔、快排、简单选择、堆）。
19	快排复杂度	平均 $O(n \log n)$ ，最坏 $O(n^2)$ （已有序），空间 $O(\log n)$ 。
20	堆排建堆	从 $\lfloor n/2 \rfloor$ 开始向前 sift 调整。时间 $O(n)$ 。

2. 2026年B卷真题完整重解（附录）

以下为本手册开头给出的B卷真题的完整详细解答，每题包含答案、解题步骤、知识点定位、易错提示。

一、单项选择题（每题2分，共20分）

1. 下列数据结构中，()是线性数据结构。

【答案】D 【解析】广义表是线性表的推广，逻辑结构属线性结构。二叉树、图、树均为非线性。

2. 在n个元素的顺序表中删除第i个元素，平均移动次数为()。

【答案】B 【解析】删除第i个需移动n-i个元素。等概率下 $ASL = \sum (n-i)/n = (n-1)/2$ 。

3. 进栈元素1,2,3,4，可能出栈序列有()种。

【答案】D 【解析】卡特兰数 $C_4 = (1/5) \times C(8,4) = (1/5) \times 70 = 14$ 。

4. 模式串ABAABCA的next数组()。

【答案】A 【解析】严蔚敏版： $j=1 \rightarrow 0, j=2 \rightarrow 1, j=3 \rightarrow 1, j=4 \rightarrow 2$ (ABA最长相等前后缀A长1), $j=5 \rightarrow 2$ (ABAA最长相等A长1), $j=6 \rightarrow 3$ (ABAAB最长相等AB长2), $j=7 \rightarrow 1$ 。结果0112231。

5. 广义表C=(a,(b,c,d))，表头和表尾为()。

【答案】B 【解析】 $head(C)=a$ （第一个元素）； $tail(C)=((b,c,d))$ （剩余组成新表，加括号）。

6. 深度为k的满二叉树有()个结点。

【答案】A 【解析】满二叉树结点 $= 2^k - 1$ ($2^0 + 2^1 + \dots + 2^{(k-1)}$)。

7. n个结点二叉树的二叉链表中有()个空指针。

【答案】C 【解析】2n个指针域，n-1非空（每非根被指一次），空 $= 2n - (n-1) = n+1$ 。

8. n个顶点m条边无向图邻接表的边结点数()。

【答案】D 【解析】无向图每边在两端各出现一次，共2m个边结点。

9. 分块有序的静态查找表，最合适的查找方法是()。

【答案】B 【解析】分块有序即索引顺序表，适合分块查找（索引查找）。折半需整体有序。

10. 快速排序最坏情况下比较次数和移动次数()。

【答案】B 【解析】最坏（已有序）退化为冒泡，递归深度n，每趟O(n)，总比较和移动均O(n²)。

二、填空题（每题2分，共20分）

1. 某算法频度 $10n^2 \log n + 100n^2m/1000$ ，时间复杂度为？

【答案】O(n²log n + n²m) 或 O(n²log n)（若m为常数） 【解析】忽略常数系数，保留最高阶项。若m与n同阶，两项均保留。

2. 带头结点单链表，p指向头结点，s为待插入结点，在第一结点前插入s的核心代码？

【答案】s->next = p->next; p->next = s; 【解析】两步：①s指向原第一结点；②p指向s。顺序不能颠倒。

3. 循环队列Q.rear==Q.front是判断队列____的条件。

【答案】空 【解析】 $\text{rear} == \text{front}$ 判空； $(\text{rear} + 1) \% M == \text{front}$ 判满（牺牲一个空间）。

4. 目标串长n，模式串长m，朴素模式匹配时间复杂度？

【答案】 $O(n \times m)$ 【解析】 BF最坏每趟匹配到最后一位失败，共 n/m 趟，每趟 m 次比较。

5. $n \times n$ 对称矩阵压缩存储最少需存储____个元素。

【答案】 $n(n+1)/2$ 【解析】 下三角+对角线共 $1+2+\dots+n=n(n+1)/2$ 个元素。

6. 二叉树层序遍历适合用____结构实现。

【答案】 队列 【解析】 层序遍历逐层访问，先进先出，用队列。

7. 图遍历算法分别是____优先和____优先。

【答案】 深度；广度 【解析】 DFS深度优先；BFS广度优先。

8. n个结点平衡二叉树高度为？

【答案】 $O(\log_2 n)$ 【解析】 AVL保证高度 $O(\log n)$ 。

9. (21,25,49,5*,16,8)希尔排序gap=3一趟结果？

【答案】 (5*, 16, 8, 21, 25, 49) 【解析】 gap=3分3组：(21,5*) $\rightarrow$ (5*,21)；(25,16) $\rightarrow$ (16,25)；(49,8) $\rightarrow$ (8,49)。合并得5*,16,8,21,25,49。

10. 树形选择排序是____的排序算法。

【答案】 稳定 【解析】 树形选择排序基于比较树，相同关键字相对位置不变。

三、简答题（40分）

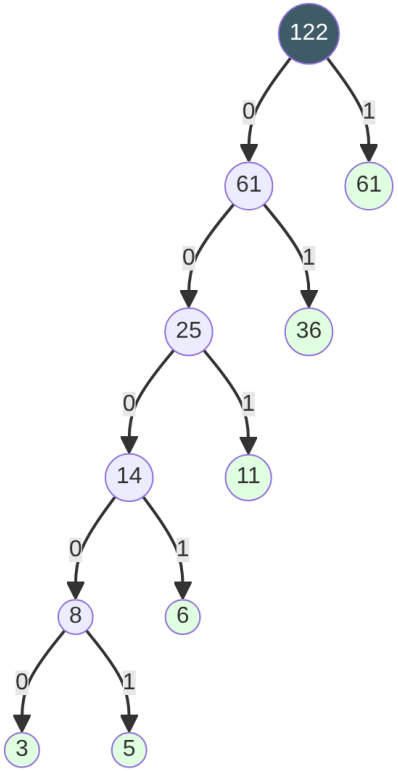
1. Huffman编码（10分）

题目：6个字母c1~c6出现次数5,61,3,6,11,36，设计Huffman编码，写出树结构、编码、总码数。

解答：

构造过程：

- 初始排序：c3(3), c1(5), c4(6), c5(11), c6(36), c2(61)
- ①合并3+5=8(N1)；序列c4(6), N1(8), c5(11), c6(36), c2(61)
- ②合并6+8=14(N2)；序列c5(11), N2(14), c6(36), c2(61)
- ③合并11+14=25(N3)；序列N3(25), c6(36), c2(61)
- ④合并25+36=61(N4)；序列N4(61), c2(61)
- ⑤合并61+61=122(Root)



附图1 哈夫曼树结构

各字母编码（左0右1）：

字母	权值	编码	码长
c2	61	0	1
c6	36	10	2
c5	11	110	3
c4	6	1110	4
c1	5	11110	5
c3	3	11111	5

总码数（WPL）：

$$WPL = 61 \times 1 + 36 \times 2 + 11 \times 3 + 6 \times 4 + 5 \times 5 + 3 \times 5 = 61 + 72 + 33 + 24 + 25 + 15 = 230$$

2. Dijkstra算法求最短路径（10分）

题目：给出带权有向图，采用Dijkstra算法求v1到其他各顶点的最短路径。

解答：依据重建图结构v1→v2:8, v1→v3:100, v2→v3:60, v2→v4:80, v3→v4:5。

邻接矩阵（∞表示无边）：

	v1	v2	v3	v4
v1	0	8	100	∞
v2	∞	0	60	80

	v1	v2	v3	v4
v3	∞	∞	0	5
v4	∞	∞	∞	0

求解过程：

步骤	选中	dist[v2]	dist[v3]	dist[v4]	S集合
初始	—	∞	∞	∞	{v1}
1	v1(0)	8	100	∞	{v1,v2}
2	v2(8)	8	$\min(100,8+60)=68$	$8+80=88$	{v1,v2,v3}
3	v3(68)	8	68	$\min(88,68+5)=73$	{v1,v2,v3,v4}

最终结果：

- v1→v2：长度**8**，路径 v1→v2
- v1→v3：长度**68**，路径 v1→v2→v3
- v1→v4：长度**73**，路径 v1→v2→v3→v4

3. 哈希表设计（10分）

题目：关键字(25,10,8,27,32,68,5,66,69)，HT[0..11]， $H(k)=k\%11$ 。分别用线性探测法和链地址法解决冲突，计算查找成功ASL。

解答：各关键字哈希地址：

$H(25)=3, H(10)=10, H(8)=8, H(27)=5, H(32)=10, H(68)=2, H(5)=5, H(66)=0, H(69)=3$

(1) 线性探测法

按顺序插入：

- 25→3（无冲突）
- 10→10（无冲突）
- 8→8（无冲突）
- 27→5（无冲突）
- 32→10冲突→探测11（成功）
- 68→2（无冲突）
- 5→5冲突→探测6（成功）
- 66→0（无冲突）
- 69→3冲突→探测4（成功）

哈希表：

位置	0	1	2	3	4	5	6	7	8	9	10	11
关键字	66	空	68	25	69	27	5	空	8	空	10	32
探测次数	1	—	1	1	2	1	2	—	1	—	1	2

$$\text{ASL成功} = (1+1+1+2+1+2+1+1+2)/9 = 12/9 = 4/3$$

(2) 链地址法

同义词链表：

- 0: 66
- 2: 68
- 3: 25 → 69
- 5: 27 → 5
- 8: 8
- 10: 10 → 32

每个链表第1个元素探测1次，第2个元素探测2次。

$$\text{ASL成功} = (1 \times 6 + 2 \times 3)/9 = 12/9 = 4/3$$

4. 堆排序 (10分)

题目：给定记录(68,57,49,65,79,19,46,13)，写出初始大顶堆，并画出二叉树形式。

解答：建堆过程（从 $i=\lfloor n/2 \rfloor=4$ 开始向前调整）：

- $i=4$ (65)：孩子19， $65>19$ ，不调整
- $i=3$ (49)：孩子46, 13，49最大，不调整
- $i=2$ (57)：孩子65, 79。最大79，交换57和79。序列变为(68,79,49,65,57,19,46,13)。此时79的孩子为空，调整结束
- $i=1$ (68)：孩子79, 49。最大79，交换68和79。序列(79,68,49,65,57,19,46,13)。68的新孩子是65和57， $68>65, 68>57$ ，调整结束

初始大顶堆：(79, 68, 49, 65, 57, 19, 46, 13)

二叉树形式（数组下标1-8对应树结构）：

```
[1]79
 / \
[2]68 [3]49
 / \ / \
[4]65 [5]57 [6]19 [7]46
 /
[8]13
```

四、算法设计题 (20分)

1. 线索二叉树遍历 (10分)

题目：已知中序线索二叉树T，写出中序后继结点查找函数 nextnode，并调用该函数写出中序遍历函数 inorder_thr。

解答：

```

// 查找中序后继结点
BiThrNode *nextnode(BiThrNode *p) {
    if (p->rtag == 1) {
        return p->rchild; // 右指针是线索，直接返回
    } else {
        p = p->rchild; // 进入右子树
        while (p->ltag == 0) // 找右子树的最左下结点
            p = p->lchild;
        return p;
    }
}

// 中序遍历线索二叉树
void inorder_thr(BiThrTree T) {
    if (T == NULL) return;
    BiThrNode *p = T;
    while (p->ltag == 0) // 找中序第一个结点(最左下)
        p = p->lchild;
    while (p != NULL) {
        visit(p); // 访问当前结点
        p = nextnode(p); // 取后继
    }
}

```

解析：nextnode分两种情况：①rtag=1，右指针是线索，直接返回；②rtag=0，右指针是孩子，后继是右子树最左下结点。inorder_thr先找最左下结点（中序首结点），然后循环调用nextnode直到NULL。整个遍历无需栈、无需递归，时间O(n)。

2. 双向链表分割（10分）

题目：已知不含头结点的双向链表，设计算法 partition，从指针p所指结点后面的位置将该双向链表分成两个双向链表，并将指针变量p调整为第二个链表的第一个结点。

解答：

```

void partition(DuLinkedList *first, DuLinkedList *p) {
    // 边界：p为空或p已是末结点，无法分割
    if (*p == NULL || (*p)->next == NULL) return;

    // 1. 获取第二个链表的第一个结点
    DuLinkedList q = (*p)->next;

    // 2. 断开：第一链表末结点next置空
    (*p)->next = NULL;
    // 第二链表头结点prior置空
    q->prior = NULL;

    // 3. 将指针p调整为第二链表的第一结点
    *p = q;
}

```

解析：核心三步：①q指向p的后继（第二链表首结点）；②p->next=NULL断开第一链表，q->prior=NULL断开第二链表与第一链表的prior链接；③通过二级指针修改p指向q（第二链表首结点）。注意：因C语言值传递无法修改外部指针，函数签名需用二级指针DuLinkedList *p，故改为

引用传递。